

METODOLOGÍA DE LA PROGRAMACIÓN PARALELA

PRÁCTICA 2

Paralelización de un algoritmo evolutivo usando paso de mensajes con MPI

Curso 2025/26

José Luis García Valverde, subgrupo 1.1

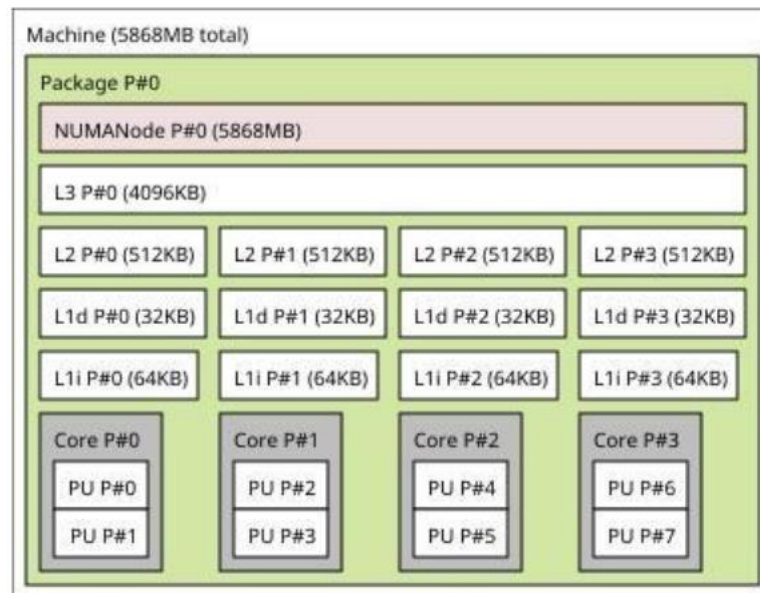
Eric Montalbán Atencia, subgrupo 1.3

Índice

Introducción.....	3
1ª Semana.....	5
Cuestión 1.	5
Cuestión 2.	9
2ª Semana.....	24
Cuestión 3.	24
Cuestión 4.	31
3ª Semana.....	42
Cuestión 5.	42
Cuestión 6.	52
Comparativa del algoritmo secuencial, paralelo con OpenMP y paralelo con MPI....	56
Conclusión.....	58

Introducción

Lo primero de todo es mostrar la información del sistema computacional en el que se han realizado las pruebas, para ello y como en la práctica 0, haremos uso del comando: `lstopo -f -p --no-legend --no-io sistema.svg`



Después hemos estimado oportuno, comentar el funcionamiento del script de bash y python que hemos creado para automatizar la experimentación y el ploteo de los resultados:

- Experimentos_npx.sh: Este script Bash automatiza la **ejecución del algoritmo evolutivo con MPI** (./par) con diferentes combinaciones de parámetros (NEM, NGM, np), repitiendo cada experimento varias veces para calcular promedios de **tiempo de ejecución** y **distancia (fitness)**. Además, con este script hemos conseguido organizar los resultados en carpetas estructuradas y generar archivos de resumen con las medias, facilitando luego la generación de gráficas y análisis de rendimiento.
- Plot_mpi_npx.py: Este script lee los ficheros resumen.txt generados por el script Bash, extrae para cada configuración (NEM, NGM) las medias de:
 - tiempo de ejecución,
 - fitness (distance_media),
 - índice de eficiencia $I = \text{distance} / \text{tiempo}$,y genera gráficas en png dentro de plots_npx/.
- Experimentos_global.sh: Este script ejecuta el programa paralelo ./par **utilizando los valores óptimos de NEM y NGM previamente seleccionados para cada número de procesos (np)**, con el objetivo de **comparar cuál de estas configuraciones proporciona el mejor comportamiento global del algoritmo en la implementación final**.
- Plot_mpi_global.py: Este script nos permite a nosotros ver de manera visual los resultados obtenidos por las mejores configuraciones para los distintos valores de np,

usando para ello tres graficas distintas; fitness vs np, tiempo_ejecucion vs np y eficiencia vs numero de procesos.

A la hora de ejecutar los scripts de bash, hemos usado el siguiente comando:

```
mpirun -np <np> --oversubscribe ./par 1000 400 200 100 0.15 <NEM> <NGM> <  
../input/input_1000_400_200_100.txt
```

Para realizar los experimentos hemos variado solamente el número de procesos con el que se ejecuta el programa y los valores del número de individuos a migrar y del número de generaciones entre migraciones.

También estimamos oportuno indicar la zona en la que hemos puesto las barreras para la toma de los tiempos, concretamente ha sido antes y después de llamar a aplicar_mh.

```
#ifdef TIME  
    double ti = 0.0;  
    if (myrank == 0) { ti = mseconds(); }  
    MPI_Barrier(MPI_COMM_WORLD);  
#endif  
  
double value = aplicar_mh(d, n, m, n_gen, tam_pob, m_rate, nem, ngm, sol, myrank, size);  
  
#ifdef TIME  
    MPI_Barrier(MPI_COMM_WORLD);  
    if (myrank == 0) {  
        double tf = mseconds();  
        printf("Execution Time: %.2lf sec\n", (tf - ti)/1000);  
    }  
#endif
```

1ª Semana

Cuestión 1.

a) Indicar qué pasos habría que llevar a cabo para paralelizar el algoritmo con MPI (no es necesario indicar las primitivas de comunicación). Puedes desarrollar la cuestión de dos maneras: (1) escribir las modificaciones que deberían realizarse sobre cada parte del código secuencial para implementar el modelo de islas, o (2) elaborar un esquema en pseudocódigo que resuma el paralelismo implementado en este modelo. En cualquier caso, tener en cuenta el patrón de computación y comunicaciones entre procesos, desde la lectura inicial de parámetros de entrada, el intercambio de información entre procesos durante el desarrollo del algoritmo, la computación en paralelo, y la comunicación final de resultados.

```
INICIAR MPI

myrank, size; // size es el número de procesos

SI (myrank == 0):
    Leer parámetros de entrada n, m, n_gen, tam_pob_m_rate, nem, ngm
    Distribuir parámetros de entrada los esclavos
    Leer matriz d
    Distribuir matriz d a los esclavos
    Crear la población inicial y enviar la parte correspondiente a cada esclavo
    Asignar subpoblación inicial en isla0

SI NO:
    Recibir del maestro n, m, n_gen, tam_pob_m_rate, nem, ngm
    Recibir del maestro d
    Recibir del maestro subpoblación inicial en isla local
FIN SI

Ordenar subpoblación inicial

PARA g desde 0 hasta n_gen - 1 HACER: // COMIENZO BUCLE EVOLUTIVO
    SI ((g > 0) Y (g % ngm == 0)): // COMIENZO MIGRACIÓN
        SI (myrank == 0):
            Recibir de los esclavos los mejores nem individuos de cada isla
            Mezclar nem * size individuos
            Ordenar nem * size individuos
            Enviar los nem mejores individuos globales a cada esclavo
            Reemplazo de los nem peores individuos locales en isla0 por los nem mejores globales

        SI NO: // ESCLAVOS
            Enviar al maestro los nem mejores individuos locales
```

```

    Recibir del maestro los nem mejores individuos globales
    Reemplazar los nem mejores globales por los nem peores locales en isla local
    FIN SI

    Ordenar isla local // Al acabar la migración
    FIN SI // FIN MIGRACIÓN

    Cruzar isla local

    Mutar isla local

    Calcular el fitness de cada individuo de la isla local

    Ordenar isla local

    FIN PARA // FIN BUCLE EVOLUTIVO

    SI (myrank == 0):
        Copiar en finalistas el mejor individuo de isla0 // finalistas[size]
        Recibir en finalistas el mejor individuo local de cada esclavo
        Ordenar finalistas
        Mostrar el mejor individuo // finalistas[0] --> solución

    SI NO: // ESCLAVOS
        Enviar mejor individuo local al maestro
    FIN SI

FINALIZAR MPI

```

b) Las primitivas como MPI_Send para comunicaciones con paso de mensajes pueden enviar arrays de datos del mismo tipo a condición de que estos estén contiguos en memoria. El código actual implementa la población como un array de punteros a Individuo con lo que los datos pueden estar en posiciones no contiguas de memoria. Como consecuencia se puede producir un error en el envío de poblaciones o subpoblaciones de individuos. Modifica el código como consideres mejor para solucionar este inconveniente, teniendo en cuenta que puedes cambiar de población de *Individuo a Individuo o reservar memoria dinámica auxiliar contigua y apuntar a ella para crear la población.

El principal objetivo de esta modificación es hacer que los datos del struct Individuos estén contiguos en memoria, que es un requisito fundamental para el uso eficiente y correcto de la primitiva MPI_Send.

En la implementación original que teníamos de Individuo, hemos reemplazado el puntero dinámico que teníamos `int *array_int` por un array de tamaño fijo lo suficientemente grande para la ejecución del programa. Este cambio nos asegura que tanto el valor fitness como el array de enteros estén contiguos en memoria.

```
#define L_MAX 4096 // MPI necesita un tamaño fijo de Individuo
eric, 4 days ago | 1 author (eric)
typedef struct {
    double fitness;
    int array_int[L_MAX];
} Individuo;
```

Dentro de la función de aplicar_mh hemos tenido que realizar una serie de cambios a la hora de la generación de la población inicial para adaptarnos a la nueva implementación del Individuo. Para ello se han eliminado las reservas de memoria dinámica y han sido reemplazadas por copia de datos. Además, también se ha modificado dentro de dicha sección como se llama a la función para el cálculo del fitness del individuo, ahora usamos `&poblacion_inicial[i]`.

```
// Generación de la POBLACIÓN COMPLETA (tam_pob)
for (int i = 0; i < tam_pob; i++) {
    int *tmp_array = crear_individuo(n, m);
    memcpy(poblacion_inicial[i].array_int, tmp_array, (size_t)m * sizeof(int));
    free(tmp_array);
    fitness(d, &poblacion_inicial[i], n, m);
}
```

También tuvimos que realizar una modificación de la función `comp_fitness` en `mh.c` para adaptar esta función al nuevo esquema del Individuo

```
int comp_fitness(const void *a, const void *b)
{
    const Individuo *ia = (const Individuo *)a;
    const Individuo *ib = (const Individuo *)b;
    // Orden descendente (mejor primero):
    if (ib->fitness > ia->fitness) return 1;
    if (ib->fitness < ia->fitness) return -1;
    return 0;
}
```

Hemos realizado la implementación del tipo derivado MPI Individuo creando la función `crear_tipo_datos` en `main.c`. Esta función tiene como objetivo principal definir el tipo de datos MPI `individuo_type`, el cual se encarga de mapear la estructura de Individuo en memoria. Esta implementación es fundamental porque le permite a MPI transferir el objeto completo de forma unificada en una sola llamada a `MPI_Send` o `MPI_Recv`. De esta forma, evitamos la complejidad de gestión de *buffers* y la sobrecarga asociada a técnicas manuales como `MPI_Pack` para consolidar el fitness y el `array_int` en un solo mensaje.

```

/**
 * Crea un nuevo tipo de datos derivado en MPI
 * Necesario para el envio/recepcion de mensajes con datos de tipo Individuo
 */
void crear_tipo_datos(MPI_Datatype *individuo_type)
{
    // Definir los 2 bloques: {1 double, L_MAX ints}
    int blocklen[2] = {1, L_MAX};

    // Definir los tipos: {MPI_DOUBLE, MPI_INT} (Fitness primero, array segundo)
    MPI_Datatype dtype[2] = { MPI_DOUBLE, MPI_INT };

    // Definir dónde empieza cada bloque (desplazamiento/offset)
    MPI_Aint disp[2];
    disp[0] = offsetof(Individuo, fitness);
    disp[1] = offsetof(Individuo, array_int);

    MPI_Type_create_struct(2, blocklen, disp, dtype, individuo_type);
    MPI_Type_commit(individuo_type);
}

```


Cuestión 2.

Paralelizar el algoritmo evolutivo utilizando primitivas de comunicación síncrona. Hacer uso de las constantes `MPI_ANY_SOURCE` o `MPI_STATUS_IGNORE`, si se considera conveniente, para evitar que el proceso 0 quede ocioso durante la recepción de mensajes del resto de procesos. Analizar las prestaciones que se obtienen al variar el número de procesos. Tener en cuenta el punto (3) en Observaciones para determinar los valores de NEM y NGM.

La implementación del Algoritmo Genético se ha realizado siguiendo una metodología de **procesamiento distribuido** y **paso de mensajes** bajo un modelo **Maestro/Esclavo**. El algoritmo opera bajo un esquema de **Modelo de Islas**, donde cada proceso gestiona una subpoblación de forma autónoma.

1. Configuración e Inicialización (Fichero main.c)

El fichero main.c se encarga de la configuración inicial del entorno paralelo, la gestión de parámetros de entrada y la distribución de estos.

1.1. Contexto MPI y Tipo de Datos Derivado

La ejecución comienza con la inicialización del entorno paralelo mediante **`MPI_Init`** y la declaración de variables esenciales como **`myrank`** (identificador de proceso) y **`size`** (número total de procesos). Inmediatamente, se llama a la función **`crear_tipo_datos`** y se realiza **`MPI_Type_commit`** para definir y activar el tipo de datos derivado **`mpi_tipo_individuo`**. Este tipo es crucial para asegurar la correcta transferencia de estructuras Individuo completas y contiguas en una sola llamada MPI, evitando problemas de alineación de memoria.

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <sys/time.h>
#include <mpi.h>
#include <stddef.h> // para 'offsetof'
#include "../include/io.h"
#include "../include/mh.h"

extern double aplicar_mh(const double *, int, int, int, int, double, int, int, int *, int, int, MPI_Datatype);

static double mseconds() {
    struct timeval t;
    gettimeofday(&t, NULL);
    return t.tv_sec*1000 + t.tv_usec/1000;
}
```

```

/**
 * Crea un nuevo tipo de datos derivado en MPI
 * Necesario para el envio/recepcion de mensajes con datos de tipo Individuo
 */
void crear_tipo_datos(MPI_Datatype *individuo_type)
{
    // Definir los 2 bloques: {1 double, L_MAX ints}
    int blocklen[2] = {1, L_MAX};

    // Definir los tipos: {MPI_DOUBLE, MPI_INT} (Fitness primero, array segundo)
    MPI_Datatype dtype[2] = { MPI_DOUBLE, MPI_INT };

    // Definir dónde empieza cada bloque (desplazamiento/offset)
    MPI_Aint disp[2];
    disp[0] = offsetof(Individuo, fitness);
    disp[1] = offsetof(Individuo, array_int);

    MPI_Type_create_struct(2, blocklen, disp, dtype, individuo_type);
    MPI_Type_commit(individuo_type);
}

```

1.2. Lectura y Distribución de Parámetros

El proceso **P0 (Maestro)** se responsabiliza de la lectura y validación de todos los argumentos de la línea de comandos. Una vez validados, P0 distribuye los parámetros al resto de procesos para garantizar que todos operen con la misma configuración:

- **Parámetros Enteros:** P0 consolida todos los argumentos de tipo int en un *array* auxiliar (params_int) y los envía a cada esclavo utilizando **MPI_Ssend**. Los esclavos reciben el bloque con **MPI_Recv** y asignan los valores a sus variables locales.
- **Parámetro Double:** El parámetro m_rate (de tipo double) se envía separadamente, dada su unicidad de tipo, también mediante **MPI_Ssend** y **MPI_Recv**.

El uso de **MPI_Ssend** (Envío Síncrono) asegura que P0 espere el inicio de la recepción por parte de cada proceso esclavo antes de continuar su flujo.

```

int main(int argc, char **argv)
{
    // Declaracion variables MPI
    int myrank, size;

```

```

int dest, source = 0;
MPI_Status status;

// INICIALIZACIÓN DE MPI
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &myrank);

MPI_Datatype mpi_tipo_individuo;
crear_tipo_datos(&mpi_tipo_individuo);

int n, m, n_gen, tam_pob, nem, ngm;
double m_rate;

// P0 lee los argumentos pasados desde la terminal
if (myrank == 0) {

    if(argc < 8) {
        fprintf(stderr, "Ayuda:\n");
        fprintf(stderr, " ./programa n m nGen tamPob m_rate nem ngm\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    n = atoi(argv[1]);
    m = atoi(argv[2]);
    n_gen = atoi(argv[3]);
    tam_pob = atoi(argv[4]);
    m_rate = atof(argv[5]);
    nem = atoi(argv[6]);
    ngm = atoi(argv[7]);

    // Validaciones
    if (m >= n) {
        fprintf(stderr, "Error: 'm' debe ser menor que 'n'.\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    if (m > L_MAX) {
        fprintf(stderr, "Error: m=%d excede L_MAX=%d\n", m, L_MAX);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
}

// Distribucion de los argumentos (MPI_SSEND/MPI_RECV)
int params_int[6];
int num_int_params = 6;

int tag_int = 1;
int tag_double = 2;

if (myrank == 0) {

```

```

params_int[0] = n;
params_int[1] = m;
params_int[2] = n_gen;
params_int[3] = tam_pob;
params_int[4] = nem;
params_int[5] = ngm;

for (dest = 1; dest < size; dest++) {
    MPI_Ssend(params_int, num_int_params, MPI_INT, dest, tag_int, MPI_COMM_WORLD);
    MPI_Ssend(&m_rate, 1, MPI_DOUBLE, dest, tag_double, MPI_COMM_WORLD);
}
} else {
    MPI_Recv(params_int, num_int_params, MPI_INT, source, tag_int, MPI_COMM_WORLD, &status);
    MPI_Recv(&m_rate, 1, MPI_DOUBLE, source, tag_double, MPI_COMM_WORLD, &status);

    n = params_int[0];
    m = params_int[1];
    n_gen = params_int[2];
    tam_pob = params_int[3];
    nem = params_int[4];
    ngm = params_int[5];
}

```

1.3. Distribución de la Matriz de Distancias

Solo el proceso **P0** realiza la lectura de la matriz de distancias (**d**) desde el fichero. Los procesos esclavos reservan la memoria necesaria para su copia local. P0 distribuye el *array* contiguo **d** a todos los procesos esclavos mediante **MPI_Ssend**. Cada esclavo utiliza **MPI_Recv** para obtener su propia copia de la matriz, proporcionando a cada isla acceso local a los datos de coste.

```

// P0 lee y envía la matriz d al resto de procesos
double *d = NULL;
int tam_d = (n * n - n) / 2;
int tag_d = 20;

if (myrank == 0) {
    d = read_distances(n);
    for (dest = 1; dest < size; dest++) {
        MPI_Ssend(d, tam_d, MPI_DOUBLE, dest, tag_d, MPI_COMM_WORLD);
    }
} else {
    d = (double *) malloc(tam_d * sizeof(double));
    MPI_Recv(d, tam_d, MPI_DOUBLE, source, tag_d, MPI_COMM_WORLD, &status);
}

```

2. Algoritmo Genético Distribuido (Fichero mh.c)

El fichero mh.c contiene la función **aplicar_mh**, que implementa la lógica del Modelo de Islas, centrándose en la gestión local de la población y la coordinación de la migración.

2.1. Inicialización de la Población por Islas

Al comienzo de **aplicar_mh**, cada proceso calcula el **tam_isla** correspondiente a su subpoblación. Se realiza la **reserva de memoria contigua** para el *buffer* local **isla**. El Maestro (P0) genera la población total, se auto-asigna su bloque y distribuye el resto a los esclavos utilizando **MPI_Ssend** y el tipo derivado **mpi_tipo_individuo**. Los esclavos reciben su bloque contiguo con **MPI_Recv**. Tras la distribución, todas las islas ordenan su población con **qsort** para iniciar el ciclo evolutivo desde el mejor individuo.

El maestro se quedará con los individuos sobrantes en caso de que el tamaño de la población no sea divisible entre el número de procesos. Lo ideal sería que se repartiesen equitativamente los individuos sobrantes entre todos los procesos generando así más balanceo de carga pero tampoco queríamos complicar más el algoritmo.

```
double aplicar_mh(const double *d, int n, int m, int n_gen, int tam_pob, double m_rate, int nem, int ngm, int *sol,
int myrank, int size, MPI_Datatype mpi_tipo_individuo)
{
    srand(time(NULL) + getpid());

    int dest, source = 0;
    int tag_init = 10;
    int tag_migra = 20; // Etiqueta para la migración
    MPI_Status status;

    int tam_isla_esclavo = tam_pob / size;
    int tam_isla; // Tamaño de la isla de este proceso. Es distinto para p0 y el resto de procesos

    if (myrank == 0) { // P0 se queda con los individuos sobrantes
        int resto = tam_pob % size;
        tam_isla = tam_isla_esclavo + resto;
    } else {
        tam_isla = tam_isla_esclavo;
    }

    if (nem > tam_isla) {
        fprintf(stderr, "Error: nem=%d excede tam_isla=%d\n", nem, tam_isla);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    Individuo *isla = (Individuo *)malloc(tam_isla * sizeof(Individuo));
    assert(isla);

    /** INICIALIZACIÓN DE LA POBLACIÓN Y REPARTO */
    /** */
```

```

if (myrank == 0) {
    // MAESTRO (P0)
    Individuo *poblacion_inicial = (Individuo *)malloc(tam_pob * sizeof(Individuo));
    assert(poblacion_inicial);

    // Generación de la POBLACIÓN COMPLETA (tam_pob)
    for (int i = 0; i < tam_pob; i++) {
        int *tmp_array = crear_individuo(n, m);
        memcpy(poblacion_inicial[i].array_int, tmp_array, (size_t)m * sizeof(int));
        free(tmp_array);
        fitness(d, &poblacion_inicial[i], n, m);
    }

    // P0 se asigna su propia subpoblación
    memcpy(isla, poblacion_inicial, tam_isla * sizeof(Individuo));

    // Reparto de la población inicial al resto de procesos
    for (dest = 1; dest < size; dest++) {
        //Individuo *bloque_a_enviar = poblacion_inicial + (dest * tam_isla);
        Individuo *bloque_a_enviar = poblacion_inicial + tam_isla + ((dest - 1) * tam_isla_esclavo);
        MPI_Ssend(bloque_a_enviar, tam_isla_esclavo, mpi_tipo_individuo, dest, tag_init,
MPI_COMM_WORLD);
    }

    free(poblacion_inicial);
} else { // ESCLAVOS (myrank > 0)
    MPI_Recv(isla, tam_isla, mpi_tipo_individuo, source, tag_init, MPI_COMM_WORLD, &status);
    //printf("P%d: Población inicial recibida del PO\n", myrank);
    //printf("%f\n", isla[0].fitness);
}

```

2.2. Bucle Evolutivo y Migración Periódica

El ciclo evolutivo ejecuta las operaciones de cruce, mutación y evaluación de *fitness* de forma **local e independiente** en cada isla.

La **migración** ocurre cada **ngm** generaciones bajo el siguiente patrón:

- **Envío de Migrantes:** Todos los procesos identifican a los **nem** (Número de Individuos Migrantes) **mejores** de su isla y los envían a P0 mediante **MPI_Ssend**.
- **Mezcla y Selección Global (P0):** El Maestro recibe los bloques de migrantes usando **MPI_Recv** con **MPI_ANY_SOURCE**, los almacena en un *buffer* de mezcla y lo ordena. P0 selecciona a los **nem mejores individuos a nivel global**.

- **Redistribución y Reemplazo:** P0 envía estos a los esclavos con **MPI_Ssend**. Todos los procesos (Maestro y esclavos) reemplazan a sus **nem peores individuos** con los recién llegados, asegurando la inyección de calidad en la población, y reordenan su isla.

```
// BUCLE EVOLUTIVO
for (int g = 0; g < n_gen; g++)
{
    // Migramos si se requiere
    if (g > 0 && (g % ngm == 0))
    {
        Individuo *migrantes_a_enviar = isla;

        if (myrank == 0) {
            // MAESTRO (P0): Recibe, Mezcla, Ordena y Redistribuye
            int total_migrantes = nem * size;
            Individuo *buffer_mezcla = (Individuo *)malloc(total_migrantes * sizeof(Individuo));
            assert(buffer_mezcla);

            // Copiar los NEM mejores de P0 en buffer_mezcla
            memcpy(buffer_mezcla, migrantes_a_enviar, nem * sizeof(Individuo));

            // Recibimos size-1 bloques de migrantes en cualquier orden
            // Con MPI_ANY_SOURCE p0 no recibe en orden de id de los esclavos sino en el orden en que estos
envían
            for (int i = 1; i < size; i++) {
                MPI_Recv(&buffer_mezcla[i * nem], nem, mpi_tipo_individuo, MPI_ANY_SOURCE, tag_migra,
MPI_COMM_WORLD, &status);
                //printf("P0: Bloque de migrantes recibido del proceso %d\n", status.MPI_SOURCE);
            }

            // Ordenación
            qsort(buffer_mezcla, total_migrantes, sizeof(Individuo), comp_fitness);

            // Enviamos al resto de procesos los NEM mejores individuos globales del buffer_mezcla
            for (dest = 1; dest < size; dest++) {
                MPI_Ssend(buffer_mezcla, nem, mpi_tipo_individuo, dest, tag_migra, MPI_COMM_WORLD);
            }

            // Reemplazo en P0 de los NEM peores por los NEM mejores globales
            memcpy(&isla[tam_isla - nem], buffer_mezcla, nem * sizeof(Individuo));

            free(buffer_mezcla);
        } else { // ESCLAVOS (myrank > 0)
            MPI_Ssend(migrantes_a_enviar, nem, mpi_tipo_individuo, 0, tag_migra, MPI_COMM_WORLD);

            // Reemplazo de los peores por los mejores globales que llegan del maestro
            MPI_Recv(&isla[tam_isla - nem], nem, mpi_tipo_individuo, 0, tag_migra, MPI_COMM_WORLD,
&status);
        }
    }
}
```

```

// Todos los procesos reordenan su población local después del reemplazo/migración
qsort(isla, tam_isla, sizeof(Individuo), comp_fitness);
}

// cruce
for (int i = 0; i < (tam_isla / 2) - 1; i += 2)
{
    cruzar(&isla[i],
          &isla[i + 1],
          &isla[tam_isla / 2 + i],
          &isla[tam_isla / 2 + i + 1], n, m);
}

// mutación
int mutation_start = tam_isla / 4;
for (int i = mutation_start; i < tam_isla; i++)
{
    mutar(&isla[i], n, m, m_rate);
}

for (int i = 0; i < tam_isla; i++)
{
    fitness(d, &isla[i], n, m);
}

qsort(isla, tam_isla, sizeof(Individuo), comp_fitness);

if (PRINT)
{
    printf("Generacion %d, Proceso %d, Fitness = %.0lf\n", g, myrank, isla[0].fitness);
}
}

```

2.3. Recolección Final de la Solución

Al finalizar el bucle evolutivo, se consolida la solución global:

- **Envío de Finalistas:** Cada proceso esclavo envía su mejor individuo local (isla[0]) a P0 mediante **MPI_Ssend**.
- **Selección Global (P0):** El Maestro recoge todos los *finalistas* en un *buffer* de tamaño size, lo ordena con **qsort** y selecciona el individuo con el mejor *fitness* global.
- **Reporte:** P0 extrae el valor del *fitness* y el *array* de la solución óptima, lo copia al *buffer* final **sol** y lo retorna como el resultado del algoritmo genético paralelo.


```

// RECOLECCIÓN FINAL: P0 RECIBE EL MEJOR INDIVIDUO DE CADA ISLA Y DE ESOS LA
// SOLUCIÓN ES EL MEJOR

double value;
int tag_final = 30;

if (myrank == 0) {
    // buffer con los mejores individuos de cada isla
    Individuo *finalistas = (Individuo *)malloc(size * sizeof(Individuo));
    assert(finalistas);

    // P0 copia su propio mejor individuo
    memcpy(&finalistas[0], &isla[0], sizeof(Individuo));

    for (int i = 1; i < size; i++) {
        MPI_Recv(&finalistas[i], 1, mpi_tipo_individuo, MPI_ANY_SOURCE, tag_final,
MPI_COMM_WORLD, &status);
    }

    // Ordena a todos los finalistas del buffer de manera que finalistas[0] es el mejor individuo
    qsort(finalistas, size, sizeof(Individuo), comp_fitness);

    // Preparamos la solución
    qsort(finalistas[0].array_int, m, sizeof(int), comp_array_int);
    memmove(sol, finalistas[0].array_int, m * sizeof(int));
    value = finalistas[0].fitness;

    free(finalistas);
} else { // ESCLAVOS (myrank > 0)
    MPI_Ssend(&isla[0], 1, mpi_tipo_individuo, 0, tag_final, MPI_COMM_WORLD);

    // Ignorado en main.c
    value = isla[0].fitness;
}

if (isla) free(isla);

if (PRINT) {
    printf("Generaciones realizadas: %d\n", n_gen);
}

return value;
}

```

3. Determinar mejores valores de np, NEM y NGM

Para determinar los valores óptimos de los parámetros del modelo de islas (**NP**: número de procesos, **NEM**: número de individuos migrantes, y **NGM**: frecuencia de migración en generaciones) bajo la configuración de paso de mensajes (PM) síncrono, se utilizaron los scripts mencionados al comienzo de la memoria.

La metodología experimental se centró en la exploración de un espacio de parámetros definido:

- **Número de Procesos (NP)**: Se evaluaron las configuraciones {2, 4, 8}.
- **Frecuencia de Migración (NGM)**: Se mantuvo el mismo rango de prueba para todos los casos: {5, 10, 20, 50}.
- **Número de Individuos Migrantes (NEM)**: Se exploró el conjunto {5, 10, 12, 25, 50}, sujeto a una restricción crítica.

3.1 Restricción del Número de Migrantes (NEM)

Es fundamental que el número de individuos migrantes (**NEM**) no exceda el tamaño de la subpoblación local (tamaño de la isla). Dado que el tamaño total de la población es fijo (ejemplo: 100), el tamaño de cada isla depende directamente de NP:

$$\text{Tamaño isla} = \frac{\text{Tamaño de Población}}{\text{Número de procesos}}$$

NP	Tamaño de Isla	Restricción Máxima de NEM	Rango de NEM Válido
2	50	50	{5, 10, 12, 25, 50}
4	25	25	{5, 10, 12, 25}
8	12.5 (aprox. 12)	12	{5, 10, 12}

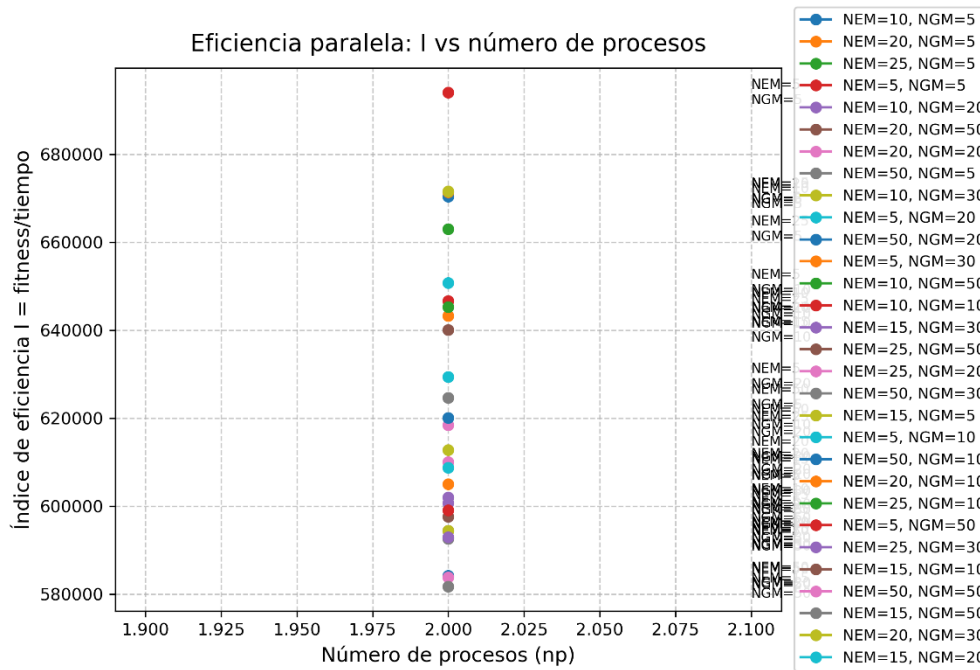
Debido a esta limitación, el rango de **NEM** se ajusta dinámicamente para cada valor de NP, mientras que el rango de **NGM** permanece constante en todas las series de ejecuciones.

Una vez ejecutado los scripts, obtenidas las medias de ejecuciones, etc. Hemos obtenido los siguientes resultados para cada uno de los np, a la hora de elegir la mejor dupla {NEM.NGM} hemos usado un índice de eficiencia:

$$I = \frac{Fitness}{Tiempo\ de\ ejecución}$$

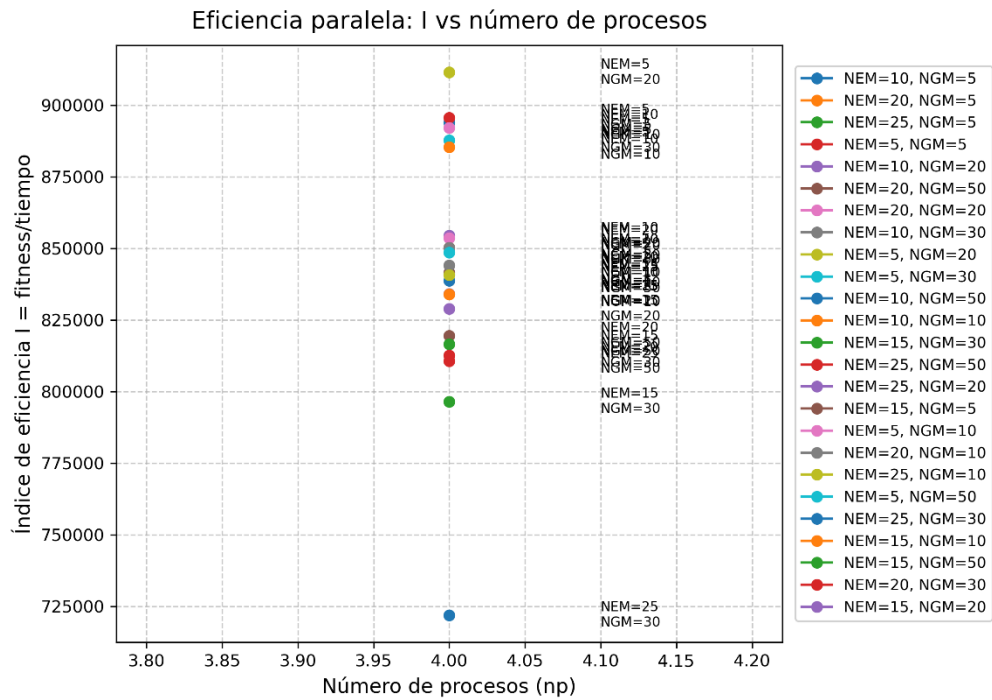
Por lo que elegiremos mejor dupla, aquella que nos dé un mayor valor de I.

Para np=2



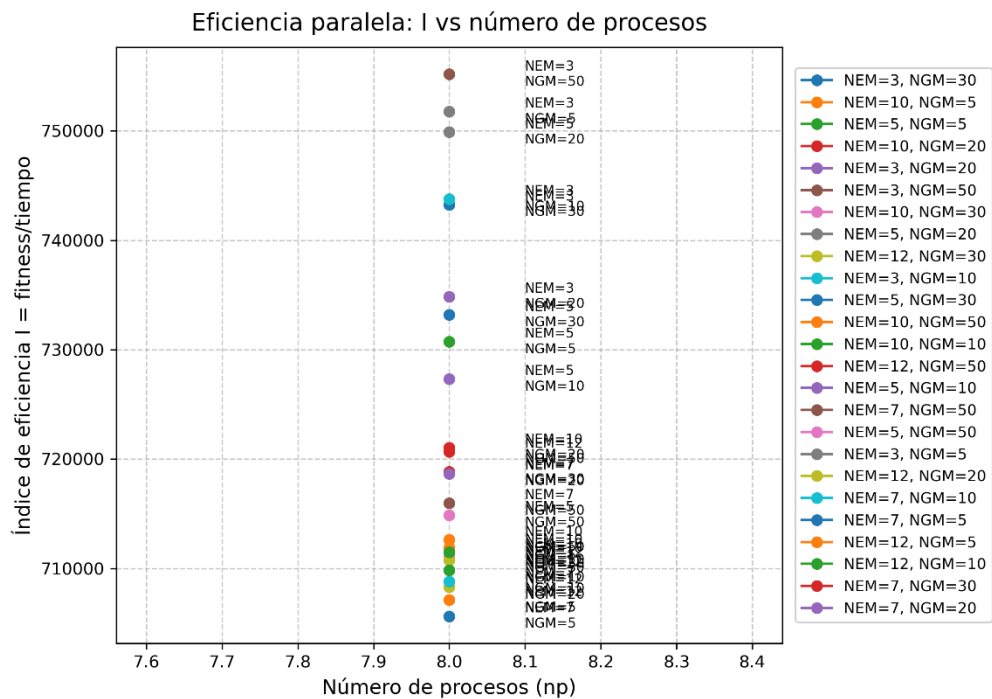
Como se puede ver en la gráfica, la dupla {NEM= 5, NGM=5} es la que mayor valor de I consigue, luego será la que usemos para compararla junto con la dupla de np=4 y np=8

Para $np=4$



Como se puede ver en la gráfica, la dupla $\{NEM=5, NGM=20\}$ es la que mayor valor de I consigue, luego será la que usemos para compararla junto con la dupla de $np=2$ y $np=8$

Para $np=8$



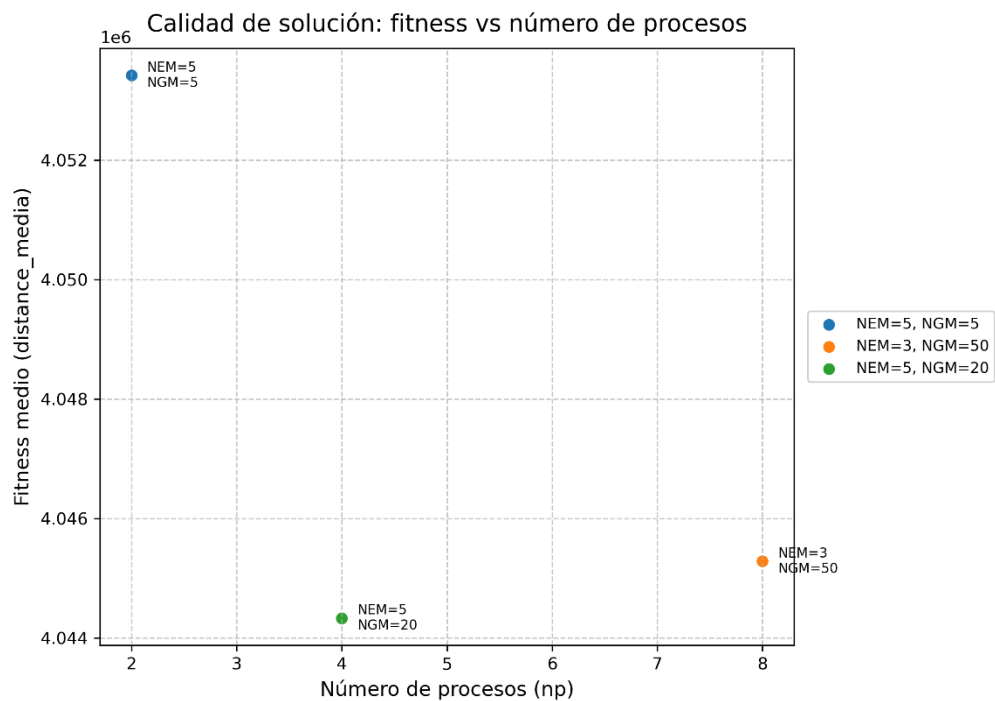
Como se puede ver en la gráfica, la dupla $\{NEM= 3, NGM=50\}$ es la que mayor valor de I consigue, luego será la que usemos para compararla junto con la dupla de $np=4$ y $np=8$.

Por último, vamos a realizar la comparativa entre las distintas configuraciones:

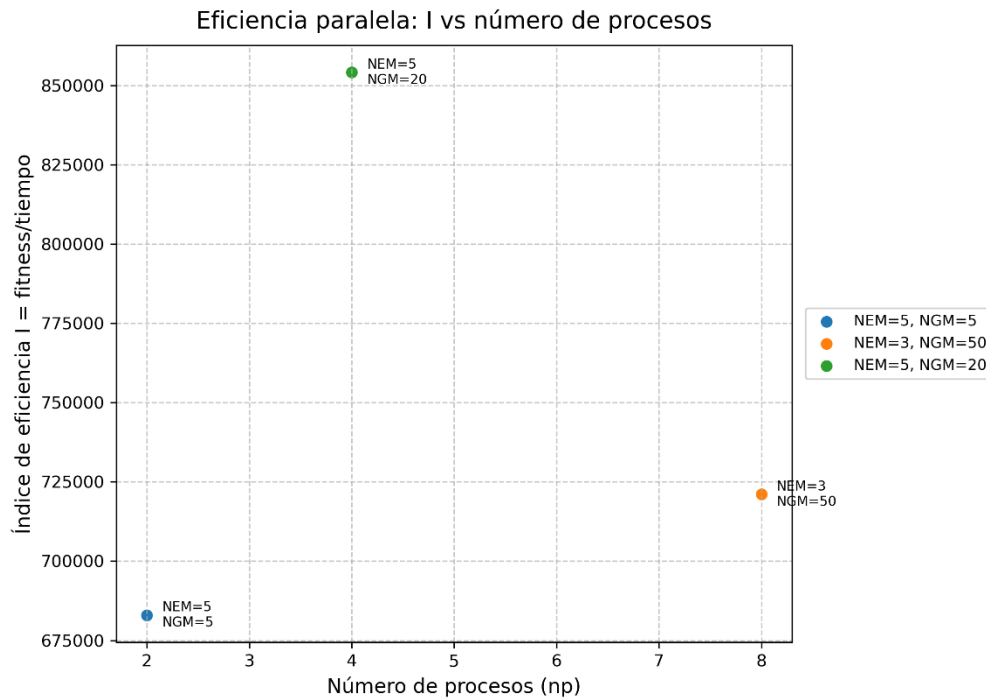
- $Np=2$, $NEM= 5$, $NGM= 5$.
- $Np=4$, $NEM= 5$, $NGM= 20$.
- $Np=8$, $NEM= 3$, $NGM= 50$.

Una vez ejecutado el script para el óptimo global y realizado el ploteo, obtenemos las siguientes tres gráficas.

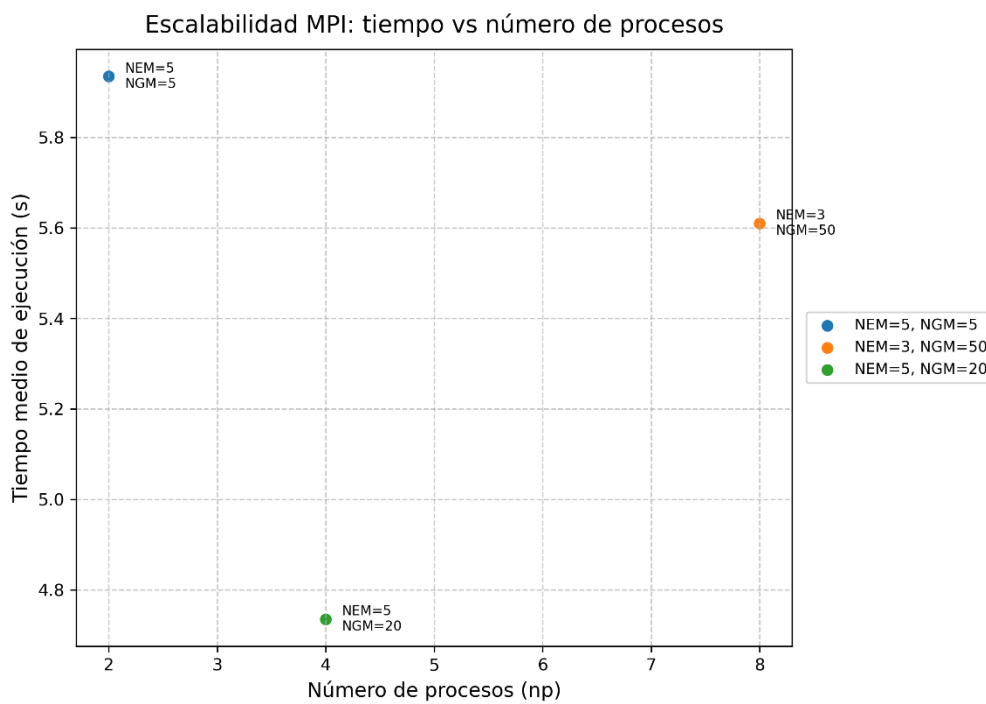
Fitness vs número de procesos



Eficiencia vs número de procesos



Tiempo ejecución vs número de procesos



El análisis de la experimentación revela una interesante compensación entre la calidad de la solución (valor de *fitness*) y la eficiencia temporal.

1. Evaluación del Rendimiento

Los resultados obtenidos son los siguientes:

- **Mejor *Fitness*:** La configuración NP=2, NEM=5, NGM=5 consigue el mejor valor de *fitness* (mayor calidad de solución). Esto pasa puesto que al tener un valor de NGM=5, es la configuración que más migra. Esto tiene como consecuencia una mayor recepción y reemplazo de individuos y por tanto obtener un mejor individuo finalista.
- **Mejor Tiempo de Ejecución:** La configuración NP=4, NEM=5, NGM=20 ofrece el tiempo de ejecución más bajo debido a que se migra relativamente poco respecto al algoritmo con 2 hilos. Por otro lado, al tener menos procesos que el algoritmo con 8 hilos se realiza un menor intercambio de mensajes y bloqueos, lo que influye drásticamente en el tiempo de ejecución.
- **Rendimiento Intermedio:** La configuración NP=8, NEM=5, NGM=30 se sitúa consistentemente entre las dos anteriores en términos de rendimiento general.

Por lo que, seleccionamos **NP=4, NEM=5, NGM=20** como la **configuración óptima**, ya que maximiza el valor del índice de eficiencia (I).

2. Justificación de la Escalabilidad no Lineal (NP=8)

Nuestra hipótesis inicial se basaba en que un mayor número de procesos (NP=8) resultaría en una ejecución más rápida debido a una mayor capacidad de cómputo paralela. Sin embargo, esta hipótesis no se ha verificado.

El rendimiento subóptimo de la configuración con **NP=8** se debe a la **penalización por comunicación**:

- **Aumento de la Sobrecarga de Comunicación:** Al duplicar el número de procesos de 4 a 8, el Maestro (P0) debe gestionar el **doble de mensajes** en las fases de migración y recolección final (8 envíos/recepciones en lugar de 4).
- **Ociosidad por Sincronización:** Dado que la implementación utiliza el envío síncrono (**MPI_Ssend**) y la recepción por parte del Maestro es un cuello de botella, un mayor número de esclavos incrementa la probabilidad de que los procesos esclavos permanezcan **ociosos**, esperando a que el Maestro esté disponible para recibir o enviar mensajes.
- **Tamaño de Isla Reducido:** El menor tamaño de las islas en NP=8 (12 individuos) reduce la diversidad poblacional local y la calidad del trabajo evolutivo en cada *isla*, mitigando cualquier beneficio de tiempo de cómputo.

2ª Semana

Cuestión 3.

Modificar la cuestión anterior para que el envío y la recepción de datos se realice mediante el uso de las funciones MPI_Pack y MPI_Unpack para empaquetamiento de datos (o cualquier otro método de envío diferente al uso del tipo derivado Individuo, de forma que se envíe en un mismo mensaje toda la estructura de individuo, no varios mensajes con cada parte). Presentar también resultados de los experimentos realizados.

Para realizar este ejercicio, nosotros hemos optado por la opción del uso de MPI_Pack y MPI_Unpack como alternativa al tipo derivado Individuo. A continuación, vamos a ver qué modificaciones hemos realizado con respecto a la versión de la cuestión 2, en la que hacíamos uso del tipo derivado Individuo. Para ello, se han realizado cambios en aquellas partes del programa donde se envían y reciben individuos, en concreto:

- Cuando P0 realiza el reparto de la población inicial.
- Cuando los procesos esclavos reciben su subpoblación inicial.
- Cuando los esclavos mandan sus mejores NEM individuos para migrar.
- Cuando P0 recibe de los esclavos sus mejores NEM individuos.
- Cuando, después de calcular los NEM mejores globales, los manda de vuelta a los esclavos para que sigan evolucionando.
- Cuando los esclavos mandan a P0 su finalista.

Para que la explicación no sea muy repetitiva, vamos a centrarnos en el ejemplo del envío de varios individuos (cuando P0 manda las subpoblaciones iniciales) y en la recepción de varios individuos (cuando los procesos esclavos reciben estas subpoblaciones).

1. Envío de la población inicial por parte de P0

El proceso P0 (maestro) genera toda la población inicial y debe enviar a cada proceso esclavo la parte que le corresponde. Para no usar el tipo derivado Individuo, lo que hacemos es empaquetar manualmente los datos en un único buffer antes de enviarlos.

Los pasos que se siguen en P0 son los siguientes:

1. **Preparar el buffer de empaquetado:** Se reserva un buffer (buffer_pack) con tamaño suficiente para guardar todos los individuos que se enviarán a un proceso.

```
char *buffer_pack = (char *)malloc(buffer_size_bytes);  
assert(buffer_pack);  
int position;
```

La variable position es un desplazamiento interno que indica en qué punto del buffer se van guardando los datos. MPI la actualiza automáticamente tras cada llamada a MPI_Pack.

2. **Seleccionar el bloque de individuos a enviar:** Para cada proceso destino dest, el maestro localiza su bloque dentro del array global de población:

```
Individuo *bloque_a_enviar = poblacion_inicial + tam_isla + ((dest - 1) *
tam_isla_esclavo);
```

3. **Empaquetar cada individuo dentro del buffer:** Se recorren todos los individuos de ese bloque y se empaquetan sus campos en el mismo orden en que después serán desempaquetados:

```
for (int i = 0; i < tam_isla_esclavo; i++) {
    MPI_Pack(&bloque_a_enviar[i].fitness, 1, MPI_DOUBLE, buffer_pack,
(int)buffer_size_bytes, &position, MPI_COMM_WORLD);
    MPI_Pack(bloque_a_enviar[i].array_int, L_MAX, MPI_INT, buffer_pack,
(int)buffer_size_bytes, &position, MPI_COMM_WORLD);
}
```

Lo que se hace en este paso es:

- Copiar el valor indicado (un double o un bloque de int) al buffer.
- Convertir los datos a un formato portable.
- Avanzar automáticamente la posición para dejar espacio a los siguientes datos.

Cuando termina el bucle, la variable position contiene el número total de bytes empaquetados.

4. **Enviar el bloque empaquetado en un único mensaje:** Una vez llenado el buffer, se envía de una sola vez al proceso destino:

```
int packed_size = position;
// Envío de buffer_pack
MPI_Ssend(buffer_pack, packed_size, MPI_PACKED, dest, tag_init,
MPI_COMM_WORLD);
```

De esta forma, cada proceso esclavo recibe un único mensaje que contiene toda su subpoblación inicial, sin necesidad de enviar mensajes separados para cada campo de la estructura.

2. Recepción y desempaquetado de la subpoblación por parte de los procesos esclavos:

Una vez que el proceso maestro (P0) ha enviado las subpoblaciones empaquetadas, cada proceso esclavo (myrank > 0) recibe el mensaje que contiene todos los individuos que le corresponden. Los pasos que se siguen en los esclavos son los siguientes:

1. **Recepción del mensaje empaquetado:** Cada proceso reserva un buffer para almacenar los bytes que llegan del maestro y realiza una recepción bloqueante del mensaje:

```
MPI_Recv(buffer_pack, (int)buffer_size_bytes, MPI_PACKED, source, tag_init,
MPI_COMM_WORLD, &status);
```

Aquí el esclavo recibe un único mensaje con su bloque completo de individuos empaquetados. No se reciben mensajes separados por campo o por individuo: toda la información viaja junta.

2. **Obtener el tamaño real del mensaje recibido:** El buffer reservado puede ser más grande de lo necesario, por eso se utiliza `MPI_Get_count` para saber cuántos bytes reales se han recibido:

```
int count;
MPI_Get_count(&status, MPI_PACKED, &count);
```

La variable `count` indica exactamente cuántos bytes del `buffer_pack` contienen datos válidos. De esta forma se evita desempaquetar más bytes de los que realmente se enviaron.

3. **Desempaquetado de los individuos dentro de la subpoblación local:** Se inicializa el desplazamiento `position` a 0 y se van reconstruyendo los individuos locales (`isla[i]`) en el mismo orden en que fueron empaquetados:

```
position = 0;
// Desempaquetar individuo por individuo (tam_isla veces)
for (int i = 0; i < tam_isla; i++) {
    MPI_Unpack(buffer_pack, count, &position, &isla[i].fitness, 1,
MPI_DOUBLE, MPI_COMM_WORLD);
    MPI_Unpack(buffer_pack, count, &position, isla[i].array_int, L_MAX,
MPI_INT, MPI_COMM_WORLD);
}
```

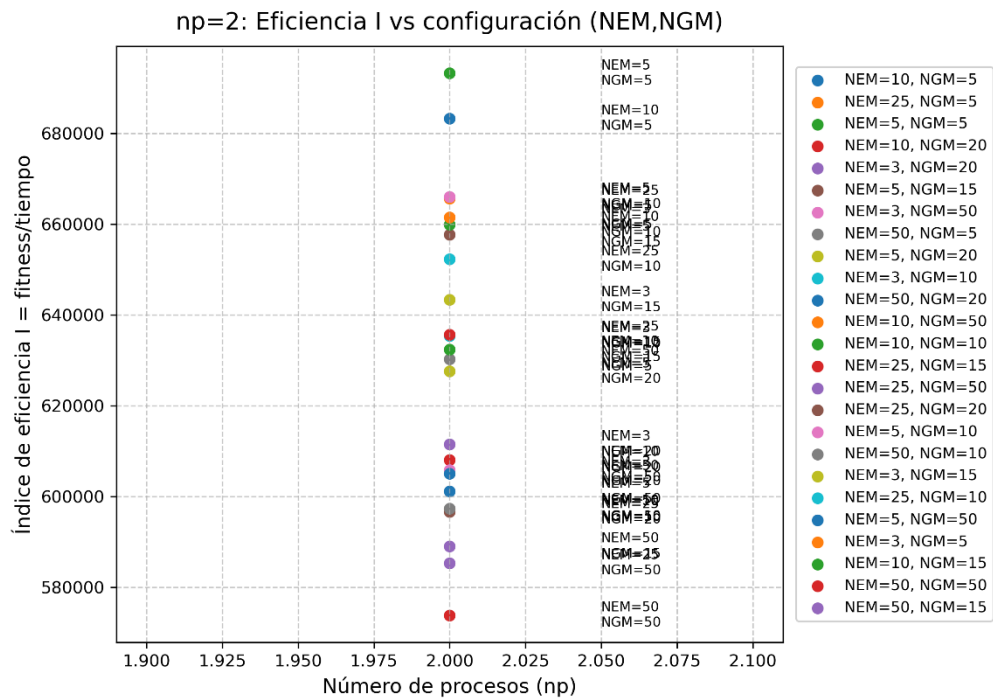
Qué ocurre aquí:

- `MPI_Unpack` lee los datos del buffer empaquetado exactamente en el orden en que fueron enviados por el maestro.
- Convierte los valores al formato local (el inverso del empaquetado).
- Actualiza la variable `position` automáticamente para apuntar al siguiente bloque de datos.

Este bucle reconstruye uno a uno todos los individuos recibidos, copiando su `fitness` y su `array_int` en la memoria del proceso esclavo.

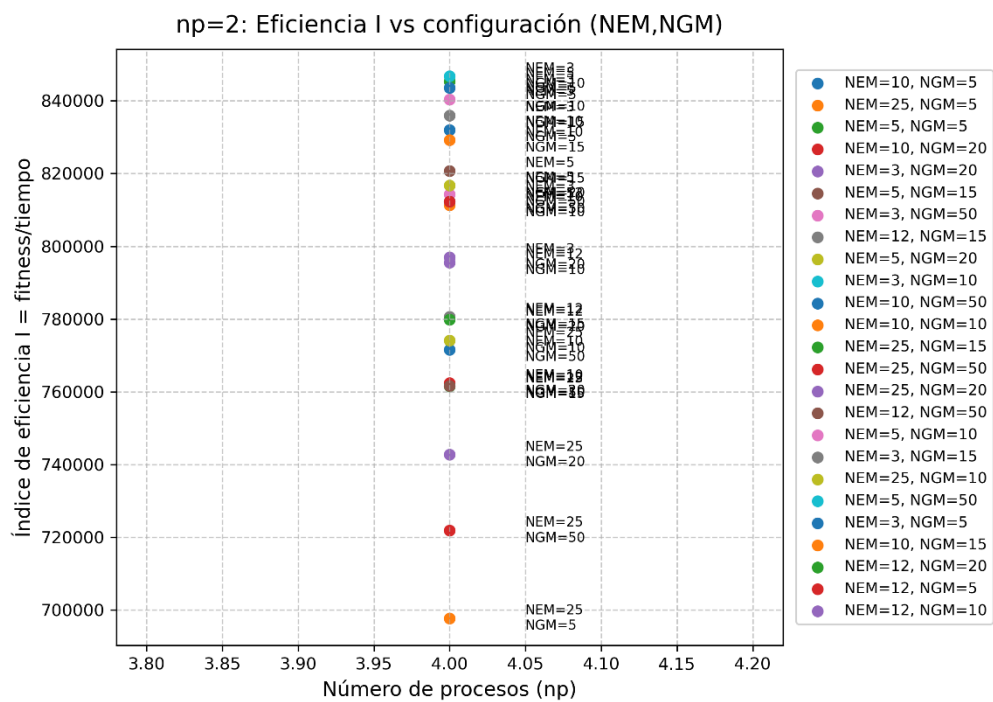
Ahora vamos a hacer los experimentos para ver cuál es el mejor conjunto de parámetros para usar en esta versión usando `MPI_Pack` y `MPI_Unpack`. Para ello vamos a seguir la misma metodología que en la cuestión 2, así que pasamos directamente a comentar las gráficas en las que vemos cuales son los mejores valores de NEM y NGM para 2, 4 y 8 procesos.

Para $np = 2$



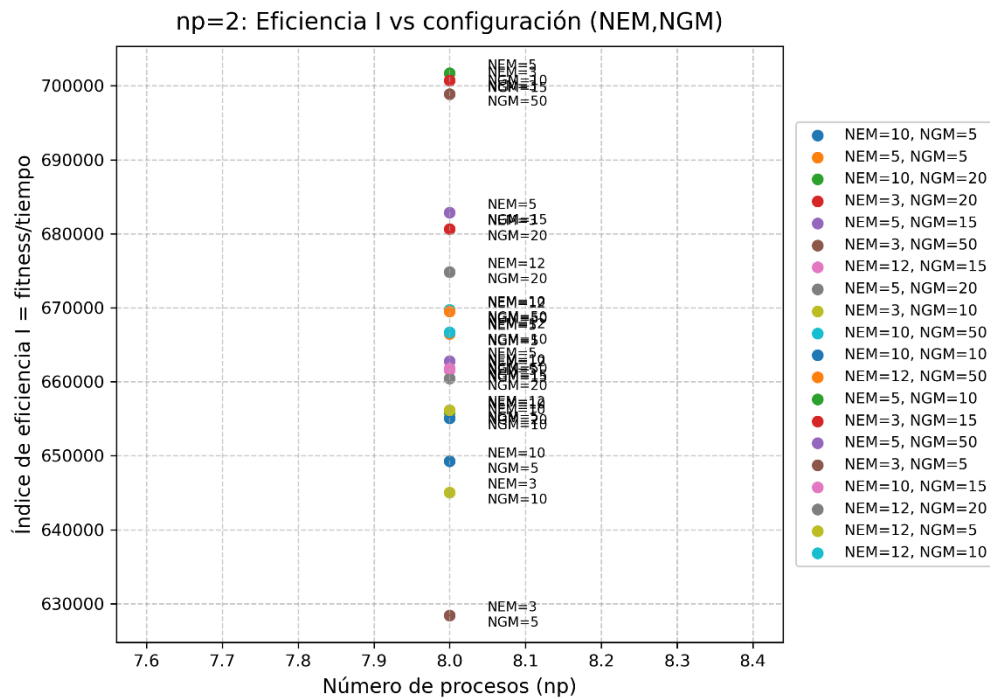
Como podemos ver en la gráfica, la mejor configuración para $np=2$ es $\{NEM = 5, NGM = 5\}$

Para $np = 4$



Como podemos ver en la gráfica, la mejor configuración para $np=4$ es $\{NEM = 3, NGM = 10\}$

Para $np = 8$

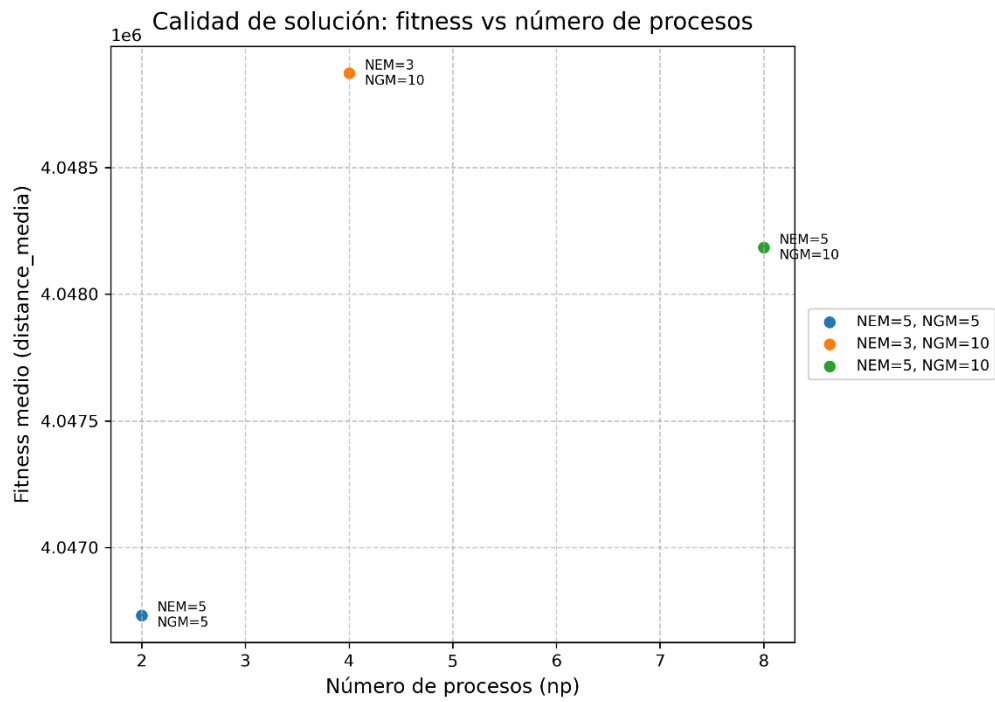


Como podemos ver para $np = 8$, la mejor configuración es $\{NEM = 5, NGM = 10\}$

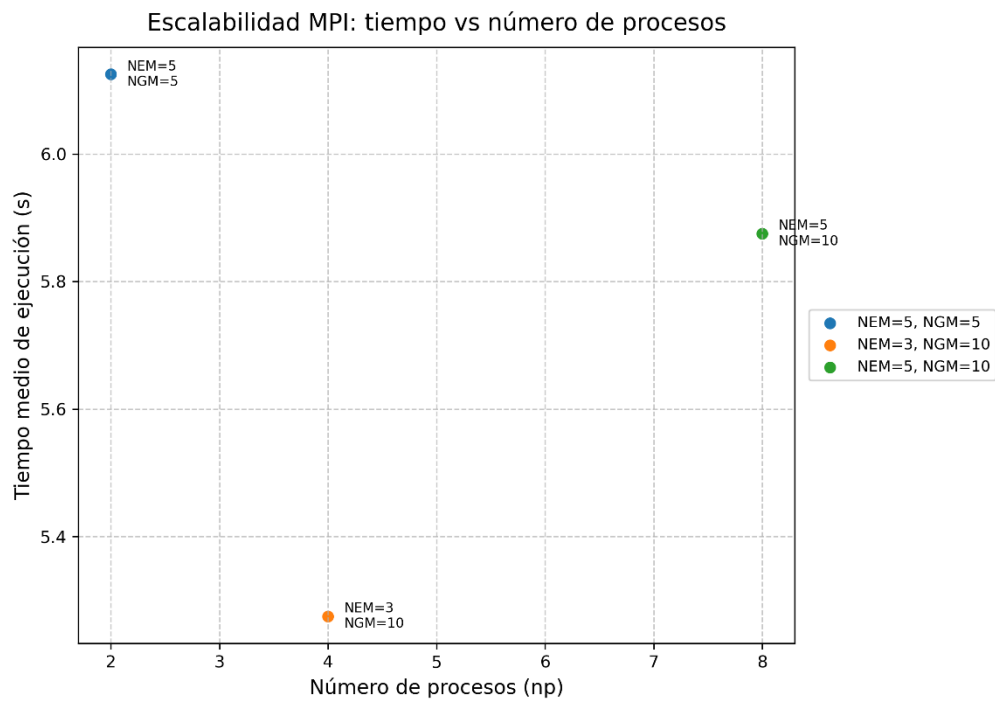
Así que ahora haremos una comparación de las distintas configuraciones para saber cuáles de las siguientes tres es la mejor:

- $Np = 2$ $NEM = 5$ $NGM = 5$
- $Np = 4$ $NEM = 3$ $NGM = 10$
- $Np = 8$ $NEM = 5$ $NGM = 10$

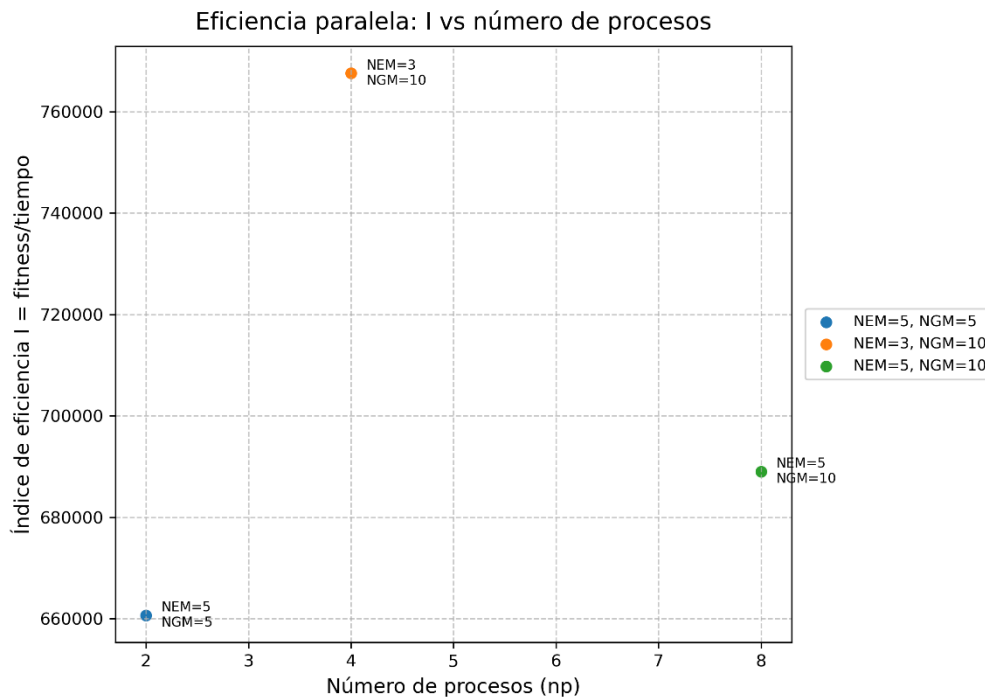
Fitness vs número de procesos



Tiempos vs número de procesos



Eficiencia vs número de procesos



Observando las gráficas de tiempo, fitness y eficiencia, se aprecia que la mejor configuración global se obtiene con **4 procesos (np=4)**, concretamente con **NEM = 3** y **NGM = 10**. Esta configuración alcanza simultáneamente el mejor índice de eficiencia y el valor de fitness más alto, lo que indica un equilibrio óptimo entre calidad de solución y rendimiento.

La razón por la que **np = 4** ofrece los mejores resultados frente a **np = 2** y **np = 8** puede explicarse de la siguiente forma:

- **Comparación con np = 2:** Con solo 2 procesos, el grado de paralelismo es bajo y cada proceso debe gestionar una subpoblación grande. Esto provoca un mayor tiempo total de ejecución, ya que la evolución dentro de cada isla tarda más en completarse.
- **Comparación con np = 8:** Con 8 procesos, aunque la carga de trabajo por proceso es menor, el coste de comunicación aumenta de forma significativa. Cada migración implica el envío y recepción de múltiples mensajes empaquetados. Esto introduce esperas entre procesos y hace que la eficiencia paralela disminuya. Además, con demasiados procesos la población se divide en subpoblaciones pequeñas, lo que reduce la variabilidad interna de cada isla y puede perjudicar la calidad del resultado final.
- **Ventaja de np = 4:** En esta configuración se alcanza un **punto de equilibrio** entre paralelismo y comunicación. Con 4 procesos hay suficiente diversidad genética y carga de trabajo por isla para mantener una evolución efectiva, y el número de mensajes entre procesos sigue siendo moderado. Las migraciones de **3 individuos cada 10 generaciones** (NEM = 3, NGM = 10) garantizan una buena difusión de soluciones sin provocar una sobrecarga de comunicación.

En conjunto, con MPI_Pack y MPI_Unpack la configuración **np = 4**, **NEM = 3**, **NGM = 10** consigue un equilibrio ideal:

- Reduce el tiempo de ejecución respecto a np = 2,
- Mantiene una comunicación eficiente sin el exceso de coste de np = 8,
- Obtiene el mejor valor de fitness, reflejando una evolución más efectiva y estable

Cuestión 4.

Paralelizar el algoritmo evolutivo utilizando primitivas de comunicación asíncrona. Analizar las prestaciones obtenidas al variar el número de procesos.

1. Configuración e inicialización (Fichero main.c)

1.1 Distribución de parámetros de entrada

```
if (myrank == 0) {
    params_int[0] = n;
    params_int[1] = m;
    params_int[2] = n_gen;
    params_int[3] = tam_pob;
    params_int[4] = nem;
    params_int[5] = ngm;

    // Se necesitan 2 envíos por esclavo (int y double)
    MPI_Request *requests = (MPI_Request *)calloc(2 * (size - 1), sizeof(MPI_Request));
    assert(requests);
    int req_count = 0;

    for (dest = 1; dest < size; dest++) {
        MPI_Isend(params_int, num_int_params, MPI_INT, dest, tag_int, MPI_COMM_WORLD,
        &requests[req_count++]);
        MPI_Isend(&m_rate, 1, MPI_DOUBLE, dest, tag_double, MPI_COMM_WORLD,
        &requests[req_count++]);
    }
    // Esperamos a que todos los envíos no bloqueantes terminen
    MPI_Waitall(req_count, requests, MPI_STATUSES_IGNORE);
    free(requests);
} else {
    // La recepción es bloqueante: el esclavo no puede hacer nada sin los params
    MPI_Recv(params_int, num_int_params, MPI_INT, source, tag_int, MPI_COMM_WORLD, &status);
    MPI_Recv(&m_rate, 1, MPI_DOUBLE, source, tag_double, MPI_COMM_WORLD, &status);

    n = params_int[0];
    m = params_int[1];
}
```

```

n_gen = params_int[2];
tam_pob = params_int[3];
nem = params_int[4];
ngm = params_int[5];
}

```

1.2 Distribución de la matriz de distancias

```

double *d = NULL;
int tam_d = (n * n - n) / 2;
int tag_d = 20;

if (myrank == 0) {
    d = read_distances(n);

    MPI_Request *d_requests = (MPI_Request *)calloc(size - 1, sizeof(MPI_Request));
    assert(d_requests);

    for (dest = 1; dest < size; dest++) {
        MPI_Isend(d, tam_d, MPI_DOUBLE, dest, tag_d, MPI_COMM_WORLD, &d_requests[dest-1]);
    }
    MPI_Waitall(size - 1, d_requests, MPI_STATUSES_IGNORE);
    free(d_requests);
} else {
    d = (double *) malloc(tam_d * sizeof(double));
    // Recepción bloqueante
    MPI_Recv(d, tam_d, MPI_DOUBLE, source, tag_d, MPI_COMM_WORLD, &status);
}

```

2. Algoritmo Genético Distribuido (Fichero mh.c)

Una vez que todos los procesos tienen los datos necesarios realizan la llamada al algoritmo evolutivo desde main.c

```
double value = aplicar_mh(d, n, m, n_gen, tam_pob, m_rate, nem, ngm, sol, myrank, size, mpi_tipo_individuo);
```

2.1. Inicialización de la población por islas

```

if (myrank == 0) {
    Individuo *poblacion_inicial = (Individuo *)malloc(tam_pob * sizeof(Individuo));
    assert(poblacion_inicial);

    for (int i = 0; i < tam_pob; i++) {
        int *tmp_array = crear_individuo(n, m);
    }
}

```



```

        memcpy(poblacion_inicial[i].array_int, tmp_array, (size_t)m * sizeof(int));
        free(tmp_array);
        fitness(d, &poblacion_inicial[i], n, m);
    }

    memcpy(isla, poblacion_inicial, tam_isla * sizeof(Individuo));

    MPI_Request *init_requests = (MPI_Request *)calloc(size - 1, sizeof(MPI_Request));
    assert(init_requests);

    for (dest = 1; dest < size; dest++) {
        Individuo *bloque_a_enviar = poblacion_inicial + tam_isla + ((dest - 1) * tam_isla_esclavo);
        MPI_Isend(bloque_a_enviar, tam_isla_esclavo, mpi_tipo_individuo, dest, tag_init,
MPI_COMM_WORLD, &init_requests[dest-1]);
    }

    MPI_Waitall(size - 1, init_requests, MPI_STATUSES_IGNORE);

    free(init_requests);
    free(poblacion_inicial);

} else { // ESCLAVOS (myrank > 0)
    // Bloqueante
    MPI_Recv(isla, tam_isla, mpi_tipo_individuo, source, tag_init, MPI_COMM_WORLD, &status);
}

```

2.2. Bucle evolutivo y migración periódica

Al ser asíncrono la filosofía del bucle evolutivo cambia un poco respecto al síncrono. La idea es que mientras el maestro recibe los *nem* mejores individuos de cada isla, mezcla, ordena y envía los *nem* mejores globales, los esclavos no permanecen bloqueados en el ámbito de la migración sino que continúan cruzando, mutando y reordenando sus respectivas subpoblaciones. De esta manera, los esclavos pueden recibir del maestro en cualquier momento los *nem* mejores globales de la última migración e incorporarlos al ciclo evolutivo. Sin embargo, la siguiente vez que toque migrar debemos comprobar que el maestro haya recibido los *nem* mejores locales de la migración anterior y que los esclavos hayan recibido del maestro.

Por su parte, el maestro, al igual que los esclavos, tampoco se bloquea a excepción de cuando debe comprobar que ha recibido en *buffer_mezcla* y enviado los *nem* mejores globales de la migración anterior. Si ésto se cumple procede a ordenar, enviar los *nem* mejores individuos globales, reemplazarlos por sus *nem* peores individuos locales y continuar con el proceso evolutivo en su isla local. En otras palabras, el maestro puede recibir en cualquier momento en *buffer_mezcla* a modo de buzón mientras prosigue cruzando, mutando y reordenando su subpoblación. No es hasta la siguiente migración cuando se debe comprobar que en *buffer_mezcla* se han recibido todos los *nem* mejores individuos locales de la migración anterior.

```

qsort(isla, tam_isla, sizeof(Individuo), comp_fitness);

MPI_Request send_request = MPI_REQUEST_NULL; // para que los esclavos envíen en la migración
MPI_Request recv_request = MPI_REQUEST_NULL; // para que los esclavos reciban en la migración
MPI_Request *send_requests = NULL;
MPI_Request *recv_requests = NULL;
bool mig_requests_pending = false; // indica si los esclavos tienen comunicaciones pendientes
bool master_requests_pending = false; // indica si el maestro tiene peticiones pendientes
Individuo *buffer_mezcla = NULL;

if (myrank == 0) {
    int total_migrantes = nem * size;
    buffer_mezcla = (Individuo *)malloc(total_migrantes * sizeof(Individuo));
    assert(buffer_mezcla);

    send_requests = (MPI_Request *)calloc(size - 1, sizeof(MPI_Request));
    recv_requests = (MPI_Request *)calloc(size - 1, sizeof(MPI_Request));
    assert(send_requests);
    assert(recv_requests);
}

// BUCLE EVOLUTIVO
for (int g = 0; g < n_gen; g++)
{
    if (g > 0 && (g % ngm == 0))
    {
        if (myrank == 0) {
            // MAESTRO (P0)
            // Esperar a que finalicen comunicaciones de la migración anterior
            if (master_requests_pending) { // No entra en la primera migración
                MPI_Waitall(size - 1, recv_requests, MPI_STATUSES_IGNORE); // nem individuos de cada
esclavo recibidos de la migración anterior
                MPI_Waitall(size - 1, send_requests, MPI_STATUSES_IGNORE); // Nos aseguramos de que el
maestro haya enviado a todos los esclavos en la migración anterior
            }

            // Procesar datos recibidos
            // En la 1era migración p0 envía sus nem mejores individuos a los esclavos, no los nem mejores globales
memcpy(buffer_mezcla, isla, nem * sizeof(Individuo));
qsort(buffer_mezcla, nem * size, sizeof(Individuo), comp_fitness);

            // Iniciar envíos para esta migración
            for (dest = 1; dest < size; dest++) {
                MPI_Isend(buffer_mezcla, nem, mpi_tipo_individuo, dest, tag_migra, MPI_COMM_WORLD,
&send_requests[dest-1]);
            }

            // Iniciar recepciones para la siguiente migración
            for (int i = 0; i < size - 1; i++) {
                MPI_Irecv(&buffer_mezcla[(i+1) * nem], nem, mpi_tipo_individuo, MPI_ANY_SOURCE,
tag_migra, MPI_COMM_WORLD, &recv_requests[i]);

```

```

    }

    master_requests_pending = true;

    // Actualizar isla local
    memcpy(&isla[tam_isla - nem], buffer_mezcla, nem * sizeof(Individuo));
    qsort(isla, tam_isla, sizeof(Individuo), comp_fitness);

} else { // ESCLAVOS (myrank > 0)

    // Esperar a que finalicen comunicaciones de la migración anterior
    if (mig_requests_pending) { // No entra en la primera migración
        MPI_Wait(&send_request, MPI_STATUS_IGNORE); // Asegurar que los nem individuos de la
migración anterior llegaron al maestro
        MPI_Wait(&recv_request, MPI_STATUS_IGNORE); // Asegurar que el esclavo recibió del maestro
en la migración anterior
    }

    // Actualizar isla local con datos recibidos
    qsort(isla, tam_isla, sizeof(Individuo), comp_fitness);

    // Iniciar envío para esta ronda
    MPI_Isend(isla, nem, mpi_tipo_individuo, 0, tag_migra, MPI_COMM_WORLD, &send_request);

    // Iniciar recepción para la próxima ronda
    // Funciona como un buzón
    MPI_Irecv(&isla[tam_isla - nem], nem, mpi_tipo_individuo, 0, tag_migra, MPI_COMM_WORLD,
&recv_request);

    mig_requests_pending = true;
}
}

for (int i = 0; i < (tam_isla / 2) - 1; i += 2)
{
    cruzar(&isla[i], &isla[i + 1], &isla[tam_isla / 2 + i], &isla[tam_isla / 2 + i + 1], n, m);
}

int mutation_start = tam_isla / 4;
for (int i = mutation_start; i < tam_isla; i++)
{
    mutar(&isla[i], n, m, m_rate);
}

for (int i = 0; i < tam_isla; i++)
{
    fitness(d, &isla[i], n, m);
}

qsort(isla, tam_isla, sizeof(Individuo), comp_fitness);

```

```

    if (PRINT)
    {
        printf("Generacion %d, Proceso %d, Fitness = %.0lf\n", g, myrank, isla[0].fitness);
    }
}

```

2.3 Cierre de las comunicaciones pendientes

Al salir del bucle evolutivo se deben cerrar las comunicaciones que hayan quedado pendientes.

```

if (myrank == 0) {
    MPI_Waitall(size - 1, recv_requests, MPI_STATUSES_IGNORE);
    MPI_Waitall(size - 1, send_requests, MPI_STATUSES_IGNORE);

    free(buffer_mezcla);
    free(send_requests);
    free(recv_requests);
} else { // Esclavo
    MPI_Wait(&send_request, MPI_STATUS_IGNORE);
    MPI_Wait(&recv_request, MPI_STATUS_IGNORE);
}

```

2.4 Recolección final de la solución

```

double value;
int tag_final = 30;

if (myrank == 0) {
    Individuo *finalistas = (Individuo *)malloc(size * sizeof(Individuo));
    assert(finalistas);
    memcpy(&finalistas[0], &isla[0], sizeof(Individuo));

    MPI_Request *final_reqs = (MPI_Request *)calloc(size - 1, sizeof(MPI_Request));
    assert(final_reqs);

    for (int i = 1; i < size; i++) {
        MPI_Irecv(&finalistas[i], 1, mpi_tipo_individuo, MPI_ANY_SOURCE, tag_final,
MPI_COMM_WORLD, &final_reqs[i-1]);
    }
    MPI_Waitall(size - 1, final_reqs, MPI_STATUSES_IGNORE);
    free(final_reqs);

    qsort(finalistas, size, sizeof(Individuo), comp_fitness);

    qsort(finalistas[0].array_int, m, sizeof(int), comp_array_int);
    memmove(sol, finalistas[0].array_int, m * sizeof(int));
    value = finalistas[0].fitness;
}

```

```

free(finalistas);

} else {
    MPI_Request final_send_req;
    MPI_Isend(&isla[0], 1, mpi_tipo_individuo, 0, tag_final, MPI_COMM_WORLD, &final_send_req);
    MPI_Wait(&final_send_req, MPI_STATUS_IGNORE);

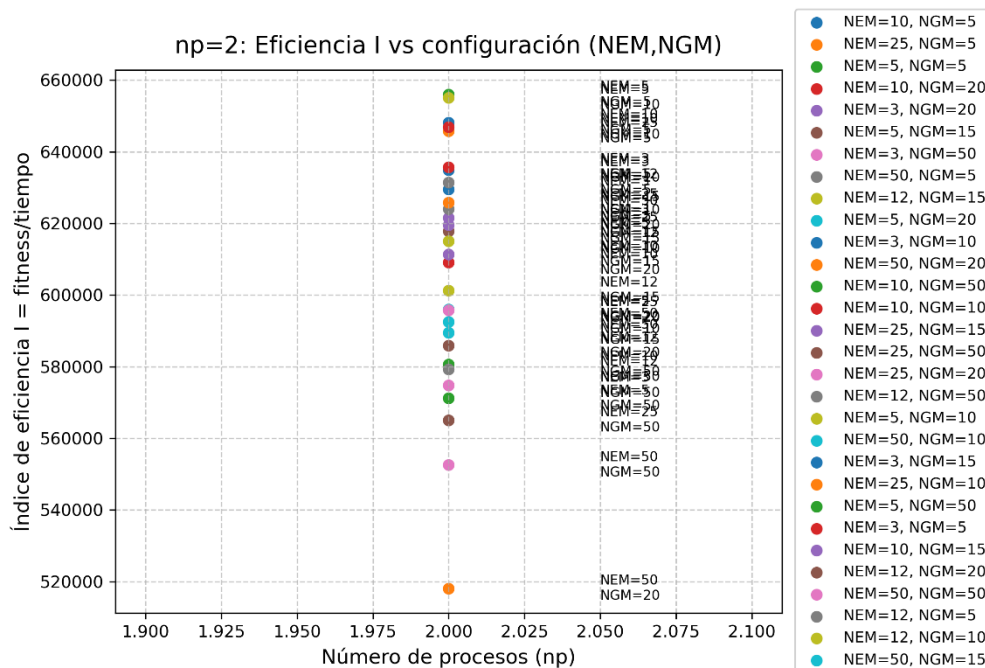
    value = isla[0].fitness;
}

```

3. Determinar mejores valores de np, NEM y NGM

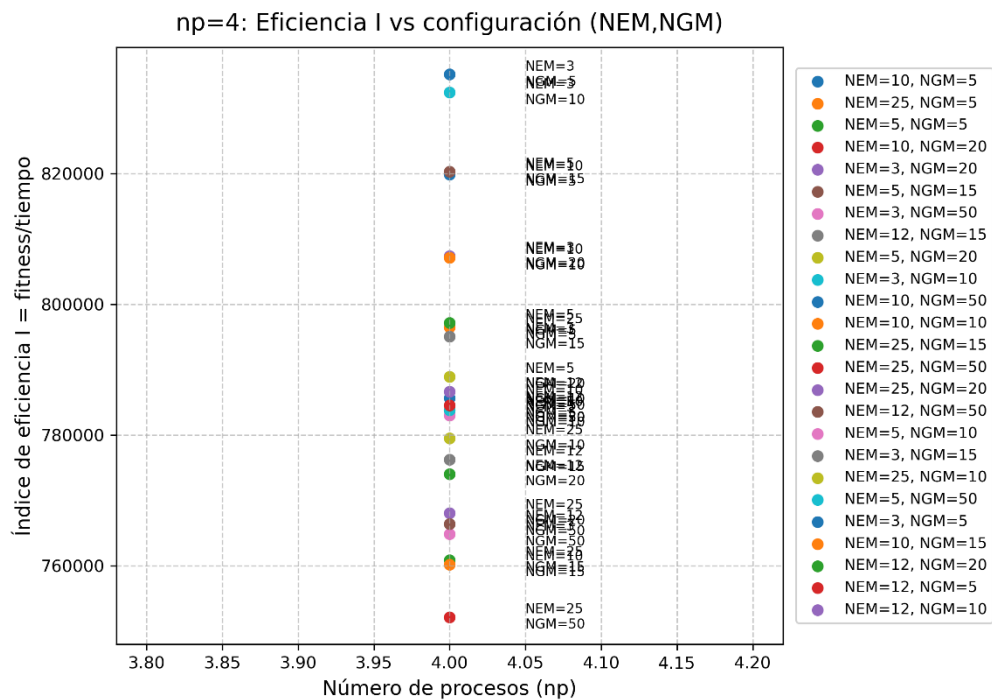
Ahora vamos a realizar los experimentos para saber para MPI asíncrono cual es la mejor configuración de parámetros. Para ello vamos a seguir la misma metodología que en las cuestiones anteriores, así que pasamos a ver las gráficas del índice de eficiencia para cada número de procesos

Para np = 2



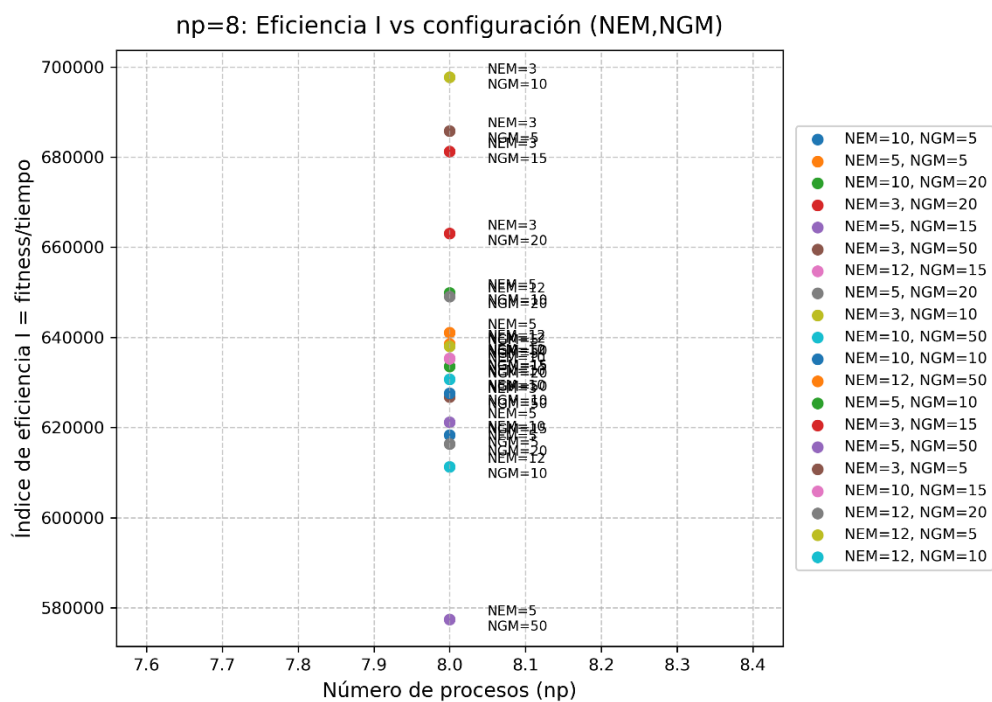
Como podemos ver en la gráfica para np = 2, la mejor configuración es {NEM= 5, NGM = 5}

Para $np = 4$



Como podemos ver para $np = 4$, la mejor configuración es $\{NEM=3, NGM=5\}$

Para $np = 8$

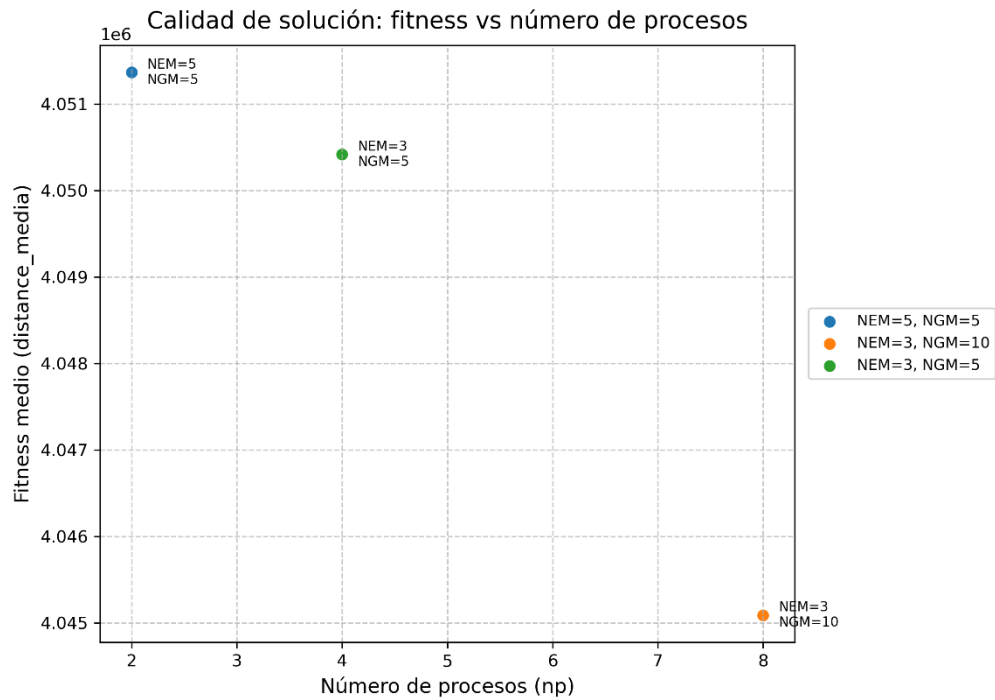


Como podemos ver para $np = 8$, la mejor configuración es $\{NEM=3, NGM=10\}$

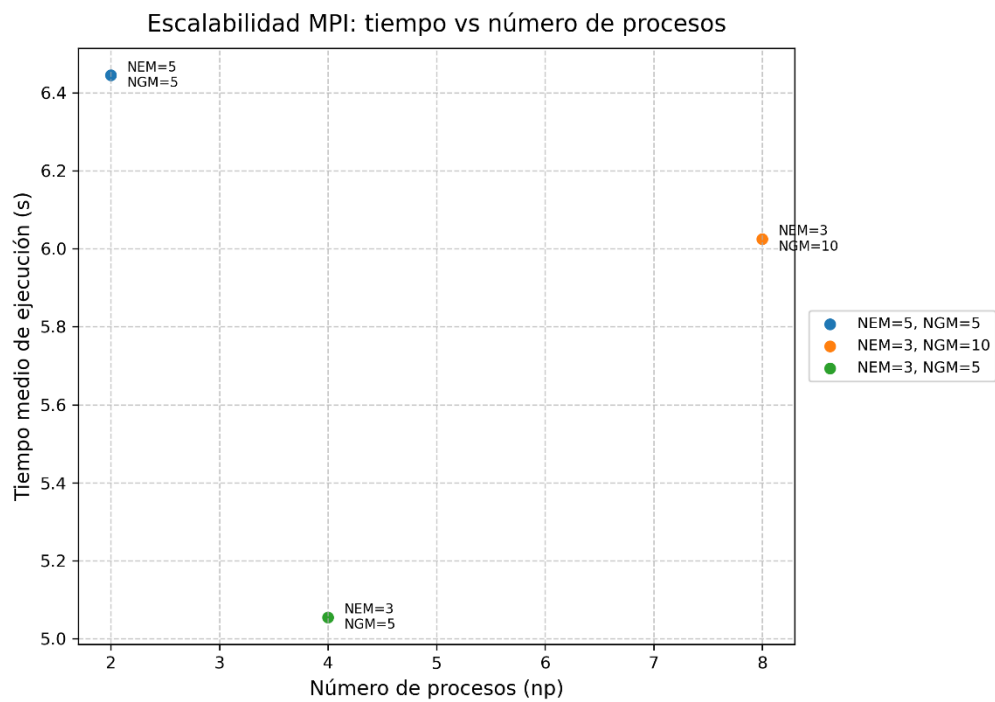
Ahora vamos a realizar los experimentos para saber cuáles la mejor de las configuraciones para la implementación de MPI asíncrona:

- $N_p = 2$ NEM = 5 NGM = 5
- $N_p = 4$ NEM = 3 NGM = 5
- $N_p = 8$ NEM = 3 NGM = 10

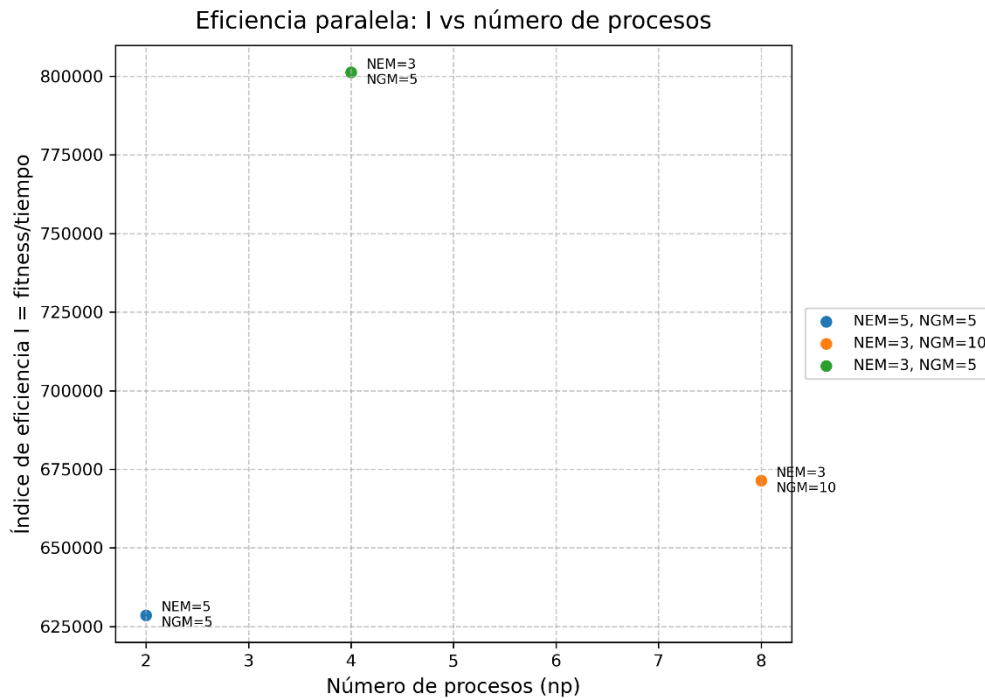
Fitness vs número de procesos



Tiempo de ejecución vs número de procesos



Eficiencia vs número de procesos



Como podemos ver, las gráficas muestran que la configuración más equilibrada y eficiente es la que utiliza **4 procesos ($np = 4$)** con **NEM = 3** y **NGM = 5**. Esta configuración consigue el **mejor valor de eficiencia** y, al mismo tiempo, mantiene una **excelente calidad de solución** con un **tiempo de ejecución reducido**, lo que la convierte en la más óptima dentro del conjunto de pruebas.

La explicación de estos resultados se basa en el comportamiento propio de la comunicación asíncrona en MPI:

- **Comparación con $np = 2$:** Con solo 2 procesos, el paralelismo es limitado y el aprovechamiento del tiempo de cómputo no es tan alto. Aunque la comunicación entre procesos es mínima, cada proceso gestiona una subpoblación demasiado grande, lo que aumenta el tiempo total de ejecución. Además, la falta de diversidad en las islas reduce la calidad del fitness obtenido.
- **Comparación con $np = 8$:** Con 8 procesos, aunque hay más paralelismo, las comunicaciones asíncronas se vuelven más costosas debido al mayor número de mensajes pendientes y solicitudes simultáneas (MPI_Isend / MPI_Irecv). Esto genera más *overhead* en la gestión de las migraciones y, en ocasiones, descompensa el rendimiento global. Además, al dividir la población total en muchas subpoblaciones pequeñas, la variabilidad dentro de cada isla se reduce, lo que puede perjudicar la convergencia hacia buenas soluciones.
- **Ventaja de $np = 4$ (NEM = 3, NGM = 5):** Esta configuración representa un **punto de equilibrio** ideal entre paralelismo, carga de trabajo y coste de comunicación. Con 4 procesos, las comunicaciones asíncronas permiten un flujo constante de migraciones sin bloquear el cálculo, aprovechando mejor el solapamiento entre

cómputo y comunicación. Además, migrar **3 individuos cada 5 generaciones** produce una presión evolutiva adecuada: las buenas soluciones se comparten con frecuencia suficiente para mantener la diversidad global, pero sin llegar a interferir excesivamente con el trabajo independiente de cada proceso.

3ª Semana

Cuestión 5.

Paralelizar el algoritmo evolutivo utilizando primitivas de comunicación colectiva. Analizar las prestaciones obtenidas al variar el número de procesos.

1.1 Lectura y Distribución de Parámetros

Usaremos broadcast para distribuir los parámetros de entrada a todos los procesos.

```
if (myrank == 0) {  
  
    if(argc < 8) {  
        fprintf(stderr, "Ayuda:\n");  
        fprintf(stderr, " ./programa n m nGen tamPob m_rate nem ngm\n");  
        MPI_Abort(MPI_COMM_WORLD, 1);  
    }  
  
    n = atoi(argv[1]);  
    m = atoi(argv[2]);  
    n_gen = atoi(argv[3]);  
    tam_pob = atoi(argv[4]);  
    m_rate = atof(argv[5]);  
    nem = atoi(argv[6]);  
    ngm = atoi(argv[7]);  
  
    // Validaciones  
    if (m >= n) {  
        fprintf(stderr, "Error: 'm' debe ser menor que 'n'.\n");  
        MPI_Abort(MPI_COMM_WORLD, 1);  
    }  
  
    if (m > L_MAX) {  
        fprintf(stderr, "Error: m=%d excede L_MAX=%d\n", m, L_MAX);  
        MPI_Abort(MPI_COMM_WORLD, 1);  
    }  
}  
  
int params_int[6];  
int num_int_params = 6;  
  
if (myrank == 0) {  
    params_int[0] = n;  
    params_int[1] = m;  
    params_int[2] = n_gen;  
    params_int[3] = tam_pob;  
    params_int[4] = nem;  
    params_int[5] = ngm;
```

```

}

// Distribución de parámetros
MPI_Bcast(params_int, num_int_params, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(&m_rate, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);

if (myrank != 0)
{
    n = params_int[0];
    m = params_int[1];
    n_gen = params_int[2];
    tam_pob = params_int[3];
    nem = params_int[4];
    ngm = params_int[5];
}

```

1.2 Distribución de la Matriz de Distancias

Lo mismo aplica para la matriz de distancias, de nuevo usamos broadcast.

```

double *d = NULL;
int tam_d = (n * n - n) / 2;

if (myrank == 0) {
    d = read_distances(n);
} else {
    d = (double *) malloc(tam_d * sizeof(double));
    assert(d);
}

// Distribución de la matriz d
MPI_Bcast(d, tam_d, MPI_DOUBLE, 0, MPI_COMM_WORLD);

```

2. Algoritmo Genético Distribuido (Fichero mh.c)

2.1 Inicialización de la población por islas

Se usan los array auxiliares *counts_envio* y *desplazamientos* para realizar el reparto de la población inicial con *Scatterv*. El primero indica cuántos elementos recibirá cada proceso, es decir, los individuos. Por su parte, *desplazamientos* define dónde comienza el bloque de cada proceso dentro del array *poblacion_inicial*. Cabe destacar que se ha usado *Scatterv* ya que el maestro recibe una subpoblación mayor en caso de que el tamaño de la población no sea divisible entre el número de procesos.

```

Individuo *poblacion_inicial = NULL;
int *counts_envio = NULL;
int *desplazamientos = NULL;

if (myrank == 0) {
    // MAESTRO (P0)
    poblacion_inicial = (Individuo *)malloc(tam_pob * sizeof(Individuo));
    assert(poblacion_inicial);

    // Generación de la POBLACIÓN COMPLETA (tam_pob)
    for (int i = 0; i < tam_pob; i++) {
        int *tmp_array = crear_individuo(n, m);
        memcpy(poblacion_inicial[i].array_int, tmp_array, (size_t)m * sizeof(int));
        free(tmp_array);
        fitness(d, &poblacion_inicial[i], n, m);
    }

    counts_envio = (int *)malloc(size * sizeof(int)); // cuántos elementos enviar a cada proceso
    desplazamientos = (int *)malloc(size * sizeof(int)); // desde dónde debe tomar cada bloque del array
    poblacion_inicial
    assert(counts_envio && desplazamientos);

    // P0 recibe tam_isla individuos
    counts_envio[0] = tam_isla;
    desplazamientos[0] = 0;

    // Los esclavos reciben tam_isla_esclavo individuos
    for (int i = 1; i < size; i++) {
        counts_envio[i] = tam_isla_esclavo;
        desplazamientos[i] = tam_isla + (i - 1) * tam_isla_esclavo;
    }

}

// P0 envía desde poblacion_inicial
// El resto reciben en isla
MPI_Scatterv(poblacion_inicial, // Buffer de envío
             counts_envio,      // cuántos elementos enviar a cada proceso
             desplazamientos,    // desde dónde debe tomar cada bloque del array poblacion_inicial
             mpi_tipo_individuo,
             isla,               // Buffer de recepción
             tam_isla,          // Cuánto recibe este proceso
             mpi_tipo_individuo,
             0,
             MPI_COMM_WORLD);

if (myrank == 0) {
    free(poblacion_inicial);
    free(counts_envio);
}

```

```

free(desplazamientos);
}

```

2.2 Bucle Evolutivo y Migración Periódica

El bucle evolutivo permanece como en las anteriores versiones a excepción del fragmento de la migración. En él, el maestro recibe en *buffer_mezcla* los migrantes de cada isla mediante la primitiva *MPI_Gather*. Posteriormente el proceso raíz realiza un broadcast para enviar los *nem* individuos mejores globales a cada proceso.

```

qsort(isla, tam_isla, sizeof(Individuo), comp_fitness);

// BUCLE EVOLUTIVO
for (int g = 0; g < n_gen; g++)
{
    // Migramos si se requiere
    if (g > 0 && (g % ngm == 0))
    {
        int total_migrantes;
        Individuo *migrantes_a_enviar = isla;
        Individuo *buffer_mezcla = NULL;    // P0 lo usa para recibir

        if (myrank == 0) {
            total_migrantes = nem * size;
            buffer_mezcla = (Individuo *)malloc(total_migrantes * sizeof(Individuo));
            assert(buffer_mezcla);
        }

        // RECOLECCIÓN de los nem mejores locales
        MPI_Gather(migrantes_a_enviar, // Buffer de envío
            nem,          // Cantidad de envío
            mpi_tipo_individuo,
            buffer_mezcla, // Buffer de recepción
            nem,          // Cantidad de recepción
            mpi_tipo_individuo,
            0,
            MPI_COMM_WORLD);

        Individuo *mejores_globales = NULL;

        if (myrank == 0) {
            qsort(buffer_mezcla, total_migrantes, sizeof(Individuo), comp_fitness);
            mejores_globales = buffer_mezcla; // P0 usará su propio buffer para el broadcast
        } else {
            // Esclavos reservan memoria para recibir el Bcast
            mejores_globales = (Individuo *)malloc(nem * sizeof(Individuo));
            assert(mejores_globales);
        }
    }
}

```

```

// DISTRIBUCIÓN de los NEM mejores globales
Bcast(mejores_globales, // P0 envía buffer_mezcla, esclavos reciben en mejores_globales
      nem, // numero de elementos en el buffer
      mpi_tipo_individuo,
      0,
      MPI_COMM_WORLD);

memcpy(&isla[tam_isla - nem], mejores_globales, nem * sizeof(Individuo));

// Liberar buffers temporales
if (myrank == 0) {
    free(buffer_mezcla); // mejores_globales apuntaba a buffer_mezcla
} else {
    free(mejores_globales);
}

qsort(isla, tam_isla, sizeof(Individuo), comp_fitness);
}

// cruce
for (int i = 0; i < (tam_isla / 2) - 1; i += 2)
{
    cruzar(&isla[i],
           &isla[i + 1],
           &isla[tam_isla / 2 + i],
           &isla[tam_isla / 2 + i + 1], n, m);
}

// mutación
int mutation_start = tam_isla / 4;
for (int i = mutation_start; i < tam_isla; i++)
{
    mutar(&isla[i], n, m, m_rate);
}

for (int i = 0; i < tam_isla; i++)
{
    fitness(d, &isla[i], n, m);
}

qsort(isla, tam_isla, sizeof(Individuo), comp_fitness);

if (PRINT)
{
    printf("Generacion %d, Proceso %d, Fitness = %.0lf\n", g, myrank, isla[0].fitness);
}
}

```

2.3 Recolección Final de la Solución

El maestro recibe en *finalistas* el mejor individuo de cada isla mediante *Gather*.

```
double value;
Individuo *finalistas = NULL; // Solo P0 lo usa

if (myrank == 0) {
    finalistas = (Individuo *)malloc(size * sizeof(Individuo));
    assert(finalistas);
}

// Todos envían su mejor individuo (isla[0]) al buffer finalistas de P0
MPI_Gather(&isla[0],
          1, // Cantidad de envío
          mpi_tipo_individuo,
          finalistas, // Buffer de recepción (solo P0)
          1, // Cantidad de recepción (por proceso)
          mpi_tipo_individuo,
          0,
          MPI_COMM_WORLD);

if (myrank == 0) {
    qsort(finalistas, size, sizeof(Individuo), comp_fitness);

    // Preparamos la solución
    qsort(finalistas[0].array_int, m, sizeof(int), comp_array_int);
    memmove(sol, finalistas[0].array_int, m * sizeof(int));
    value = finalistas[0].fitness;

    free(finalistas);
} else { // ESCLAVOS (myrank > 0)
    value = isla[0].fitness; // ignorado en main.c
}

if (isla) free(isla);

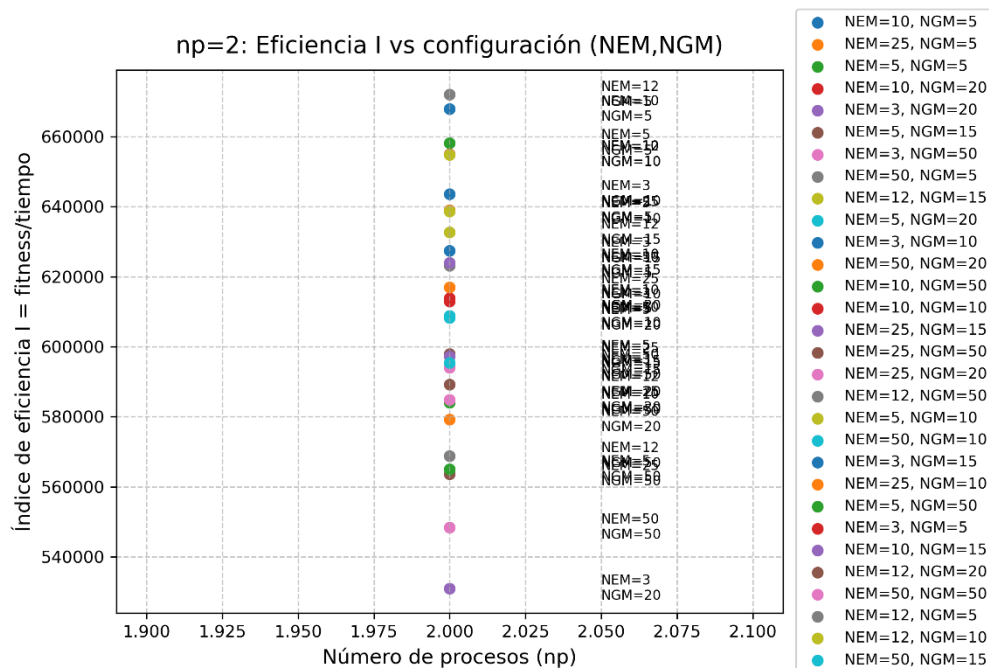
if (PRINT) {
    printf("Generaciones realizadas: %d\n", n_gen);
}

return value;
```

3. Determinar mejores valores de np, NEM y NGM

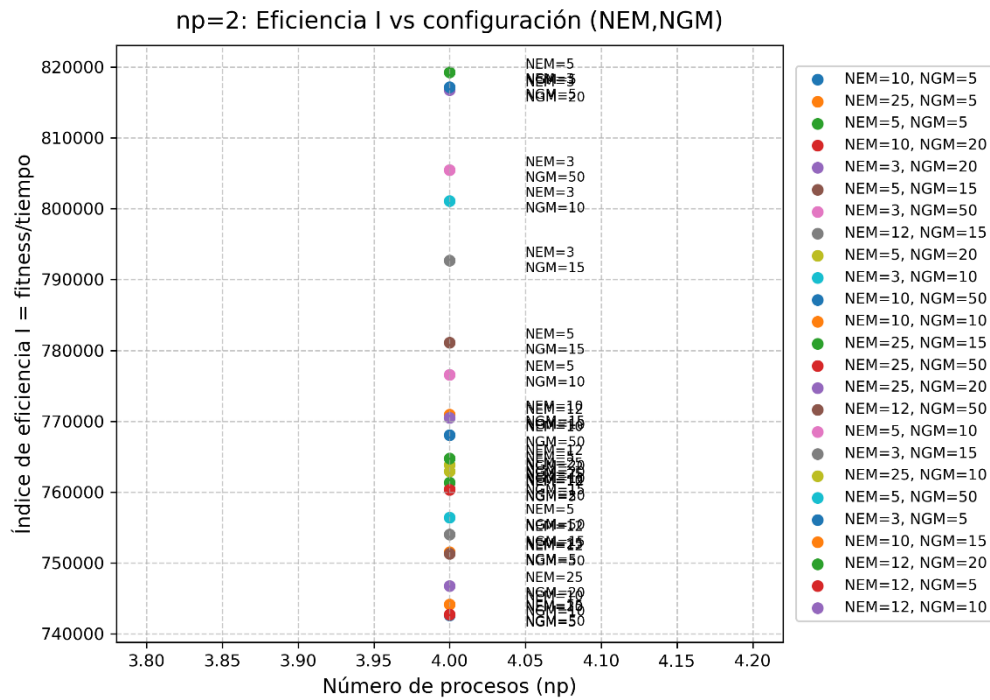
Ahora que ya hemos explicado cómo se implementa MPI usando primitivas de comunicación colectiva, pasamos a la parte de experimentación. Para ello vamos a seguir la metodología usada en todas las cuestiones anteriores: examinaremos las gráficas que muestran el valor del índice de eficiencia para las distintas posibilidades dependiendo del número de procesos.

Para np = 2



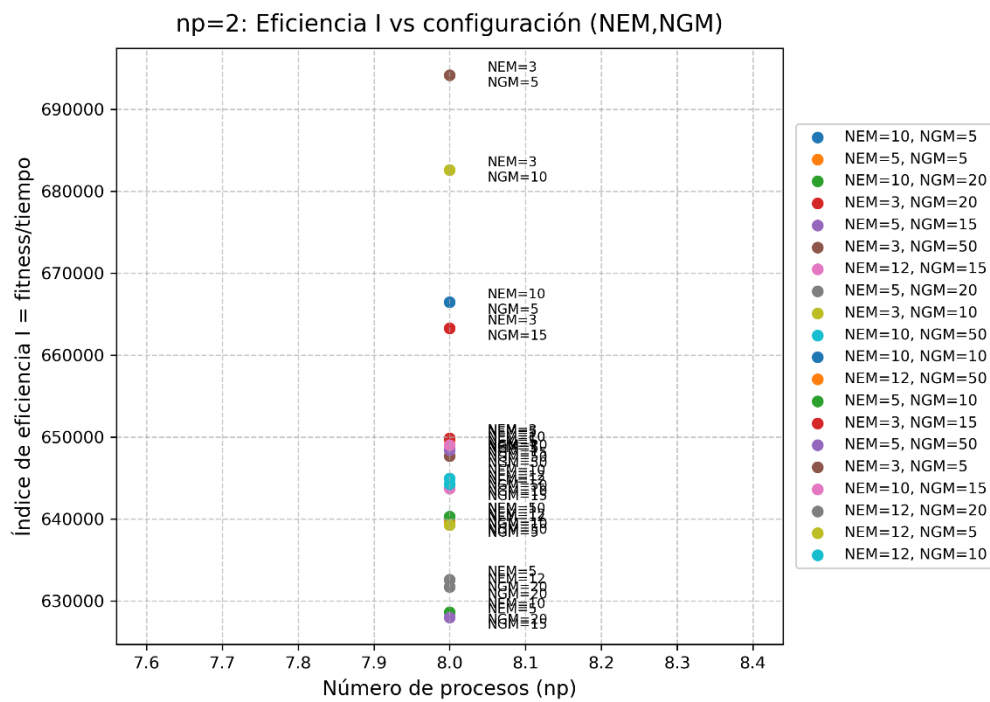
Como podemos ver en la gráfica, para np = 2, la mejor configuración es {NEM= 12, NGM= 10}

Para np = 4



Como se puede ver en la gráfica, para np = 4, la mejor configuración es {NEM=5 , NGM = 5}

Para np = 8

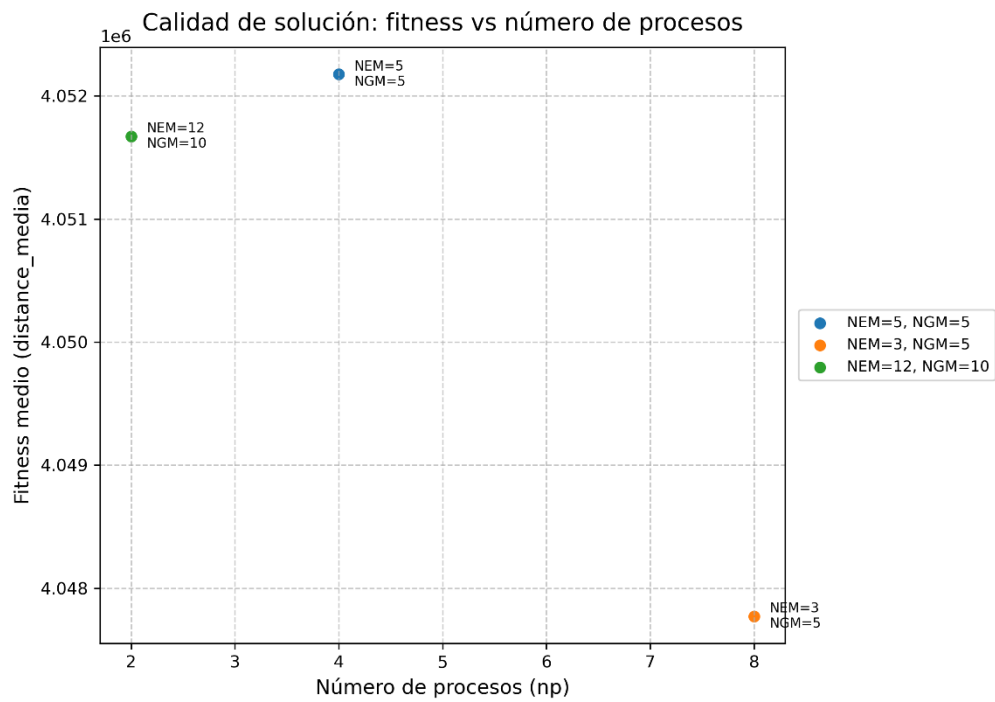


Como se puede ver en la gráfica, para np = 8, la mejor configuración es {NEM= 3, NGM = 5}

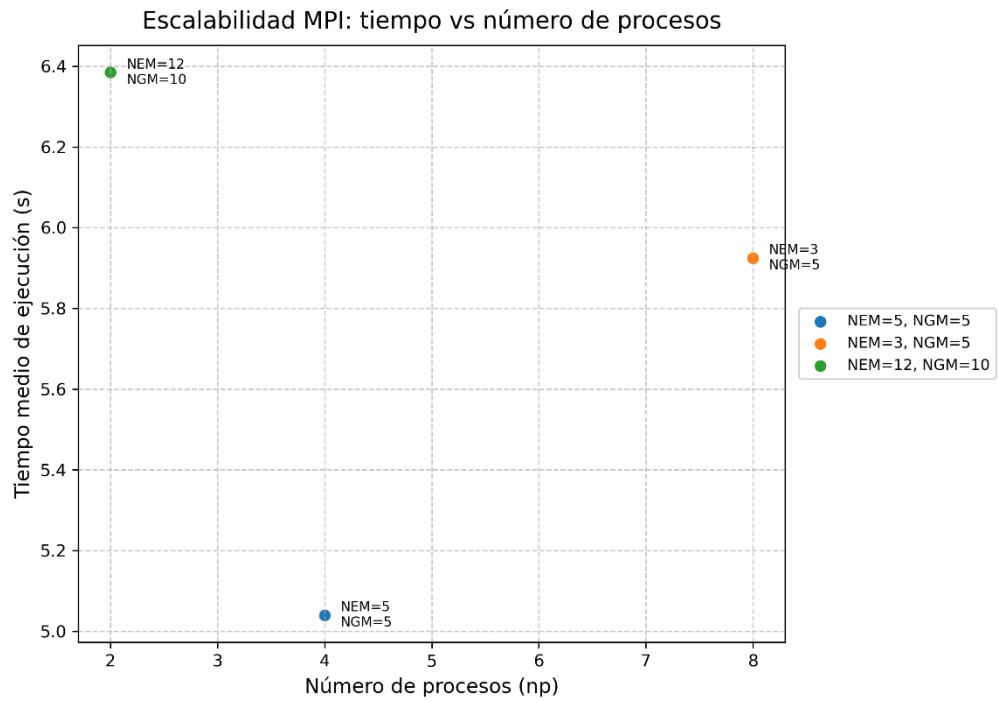
Ahora pasaremos a realizar la configuración para saber cuál de las siguientes configuraciones es la mejor:

- NP = 2 NEM = 12 NGM = 10
- NP = 4 NEM = 5 NGM = 5
- NP = 8 NEM = 3 NGM = 5

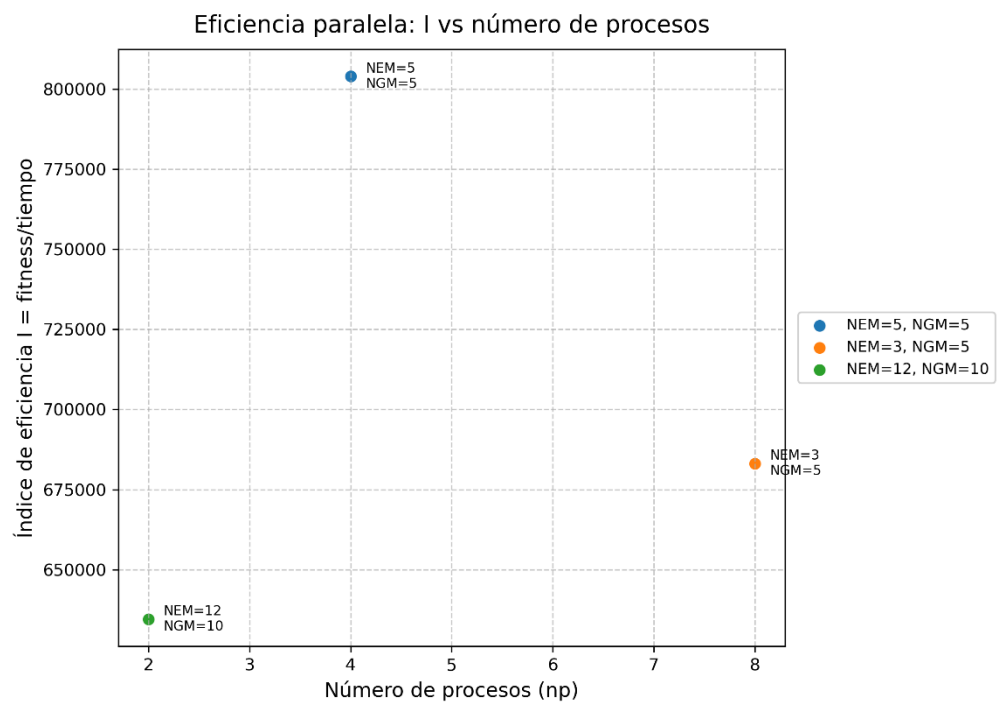
Fitness vs número de procesos



Tiempo de ejecución vs número de procesos



Eficiencia vs número de procesos



Los resultados muestran que la configuración más equilibrada y eficiente corresponde a **4 procesos con NEM = 5 y NGM = 5**. Esta combinación logra simultáneamente el **menor tiempo medio de ejecución**, el **mayor índice de eficiencia** y el **mejor valor de fitness medio**, lo que indica un excelente compromiso entre rendimiento y calidad de solución. El comportamiento observado se explica por la naturaleza sincronizada de las operaciones colectivas, donde todos los procesos deben participar en cada comunicación global, haciendo que el número de procesos influya directamente en el coste total de sincronización.

Si comparamos esta configuración con la de **2 procesos (NEM = 12, NGM = 10)**, se observa que el tiempo de ejecución es significativamente mayor. Con solo dos procesos, la carga de trabajo por proceso es más alta y la diversidad global más limitada, ya que existen menos subpoblaciones. Esto ralentiza la evolución, reduce la capacidad de escapar de óptimos locales y, en consecuencia, empeora la calidad final del fitness.

Por otro lado, al aumentar el número de procesos a **8 (NEM = 3, NGM = 5)**, el paralelismo crece, pero también lo hace el **coste de sincronización** derivado de las operaciones colectivas. En este caso, cada proceso gestiona una subpoblación muy pequeña, lo que disminuye la variabilidad interna y hace menos efectiva la evolución local. Además, las comunicaciones colectivas se vuelven más frecuentes y costosas, generando tiempos de espera entre procesos y reduciendo la eficiencia global.

Así, la configuración con **np = 4, NEM = 5 y NGM = 5** se sitúa en el punto óptimo entre ambos extremos: aprovecha el paralelismo de manera efectiva a la vez que mantiene una buena diversidad de soluciones y evita la sobrecarga de comunicación, logrando los mejores resultados tanto en tiempo como en calidad evolutiva.

Cuestión 6.

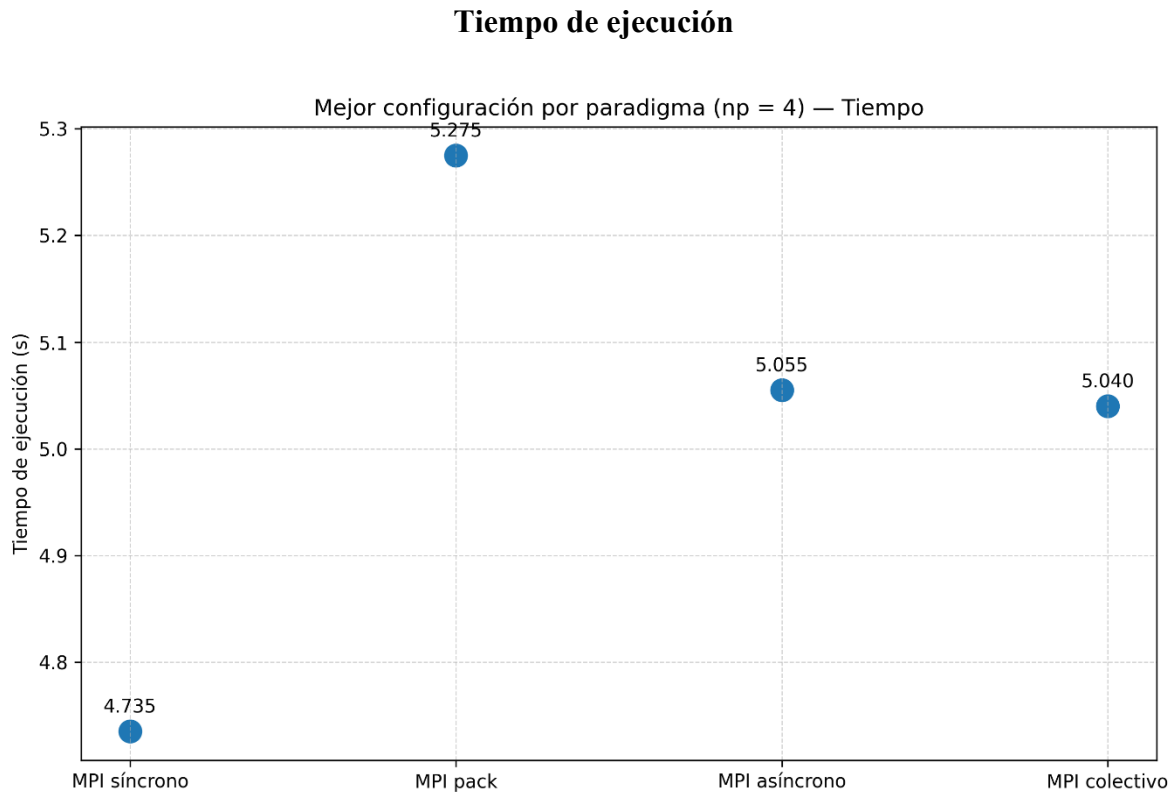
Elegir la mejor configuración de parámetros y tipo de comunicaciones para paralelizar la totalidad del algoritmo. Justificar las decisiones tomadas. De este modo, se pretende tener un código paralelo óptimo en memoria distribuida que se podrá usar para comparar con el código secuencial, y paralelo en memoria compartida e híbrido.

A la hora de elegir la mejor configuración para la implementación de nuestro algoritmo genético usando paso de mensajes con MPI, vamos a realizar una comparativa entre las mejores configuraciones de parámetros. En concreto, las configuraciones que se van a comparar son:

- MPI síncrono con tipo de dato derivado con configuración (cuestión 2):
np = 4 NEM = 5 NGM = 20
- MPI síncrono con MPI_Pack y MPI_Unpack con configuración (cuestión 3):
np = 4 NEM = 3 NGM = 10
- MPI asíncrono con configuración (cuestión 4):
np = 4 NEM = 3 NGM = 5

- MPI con comunicación colectiva con configuración (cuestión 5):
np = 4 NEM = 5 NGM = 5

Para realizar esta comparación vamos a ejecutar el script de bash para cada configuración, así que pasamos a analizar las gráficas para ver y comentar los resultados acerca de cuál es la mejor.

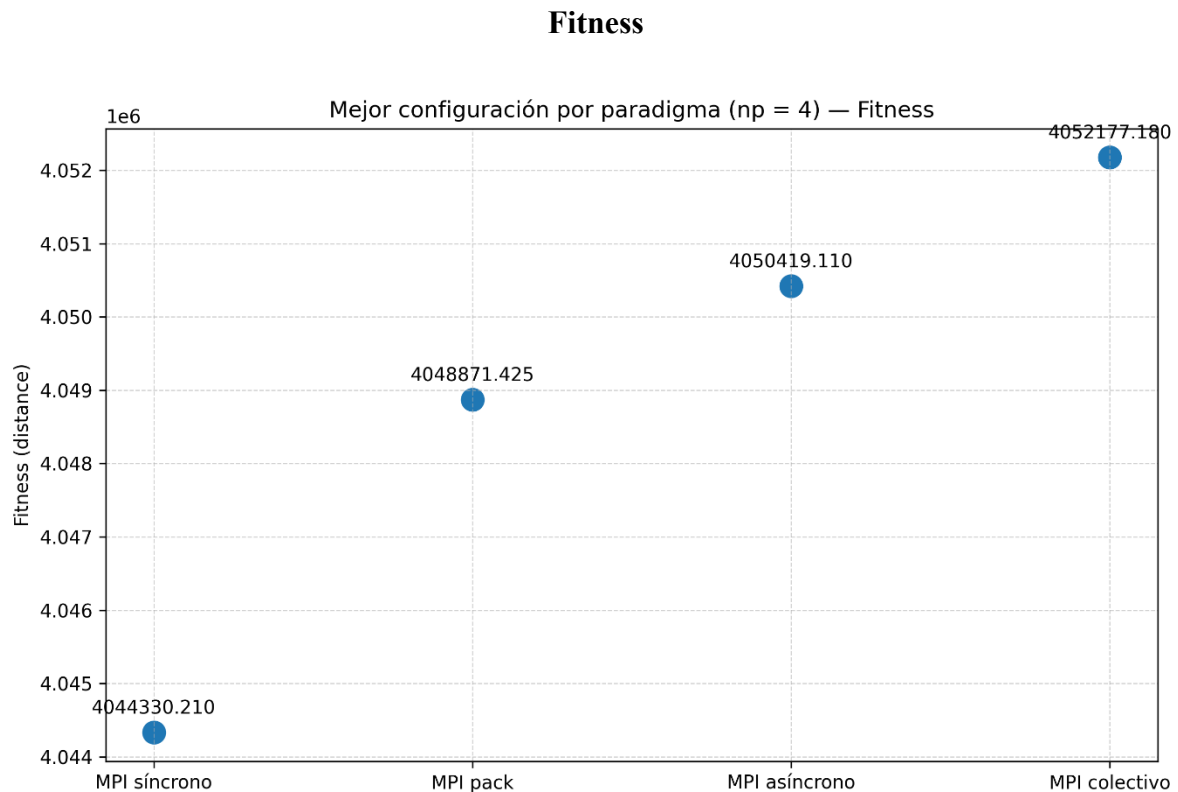


El menor tiempo de ejecución se obtiene con la versión MPI síncrona (4.735 s). Esto se debe a que la comunicación bloqueante, combinada con una frecuencia de migración baja (cada 20 generaciones) y un número moderado de individuos intercambiados (NEM = 5), reduce significativamente la sobrecarga de comunicación. Además, al emplear tipos de datos derivados, se evita el empaquetado manual de estructuras, reduciendo el tiempo de copia y simplificando las comunicaciones.

En cambio, la versión con MPI_Pack/MPI_Unpack presenta el mayor tiempo (5.275 s), lo que refleja el coste añadido de empaquetar y desempaquetar datos, especialmente en comunicaciones repetitivas.

El modelo asíncrono y el colectivo alcanzan tiempos intermedios (en torno a 5.05 s), penalizados ligeramente por la gestión adicional de *requests* o por la coordinación entre procesos en las operaciones colectivas. Pudiera parecer que la comunicación colectiva sería la más rápida por la comunicación en hipercubo paralela entre los procesos. Sin embargo, al tener

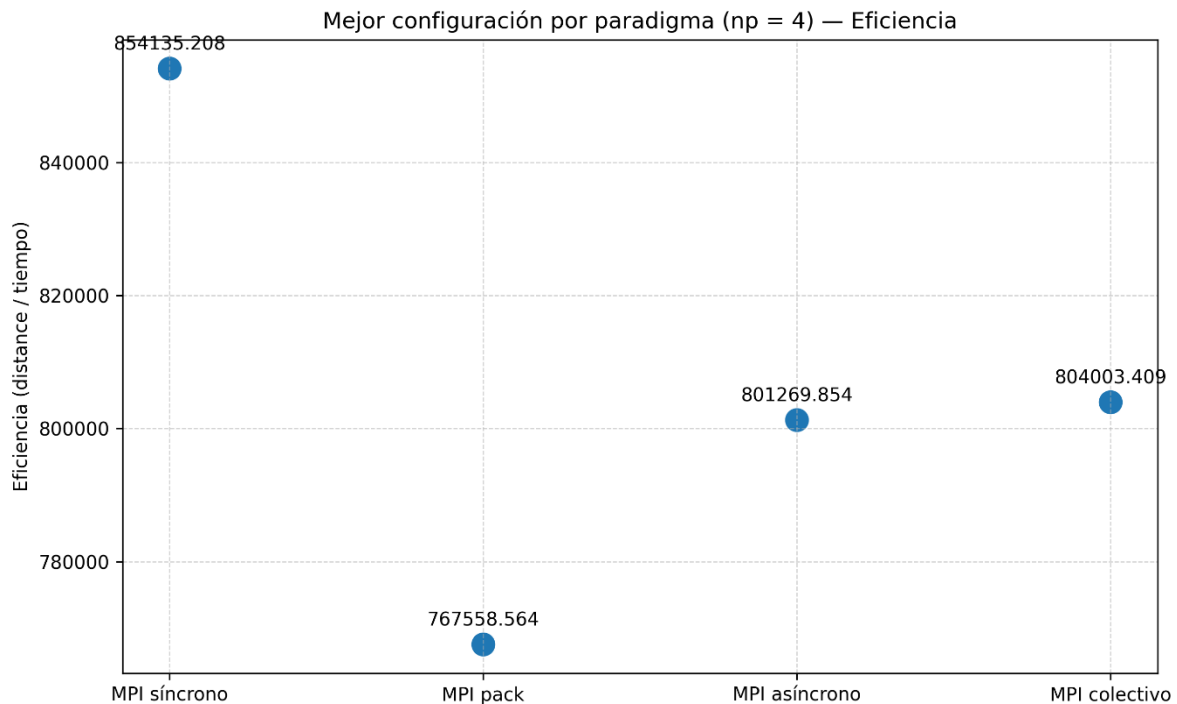
poca carga computacional junto a migrar cada 5 generaciones mientras que la versión síncrona lo hace cada 20, esta última ha sido la más rápida.



En términos de calidad de la solución (fitness o distancia), el mejor resultado lo alcanza el modelo con comunicación colectiva, con un valor de 4 052 177.18. El hecho de realizar migraciones más frecuentes (NGM = 5) y con un número de individuos moderado (NEM = 5) contribuye a un intercambio mayor de individuos, elevando el valor medio del fitness obtenido.

La configuración asíncrona ha conseguido un peor valor fitness con respecto a la colectiva, puesto que en cada una de las migraciones se intercambia un menor número de individuos, concretamente 3. No obstante, el segundo mejor fitness ha sido para esta configuración debido al alto número de migraciones (NGM = 5), mientras que las otras dos configuraciones síncronas migran cada 10 y 20 generaciones. Es por ello que hay una evidente relación entre la proporcionalidad directa de NGM y el fitness.

Eficiencia



Al analizar la relación entre calidad y tiempo de ejecución, la eficiencia es mayor en la versión MPI síncrona con tipo derivado, con un índice aproximado de 8.54×10^5 . Esto se debe a que, aunque no consigue el mejor fitness, su tiempo es considerablemente inferior, lo que le proporciona una mejor relación de bondad, I .

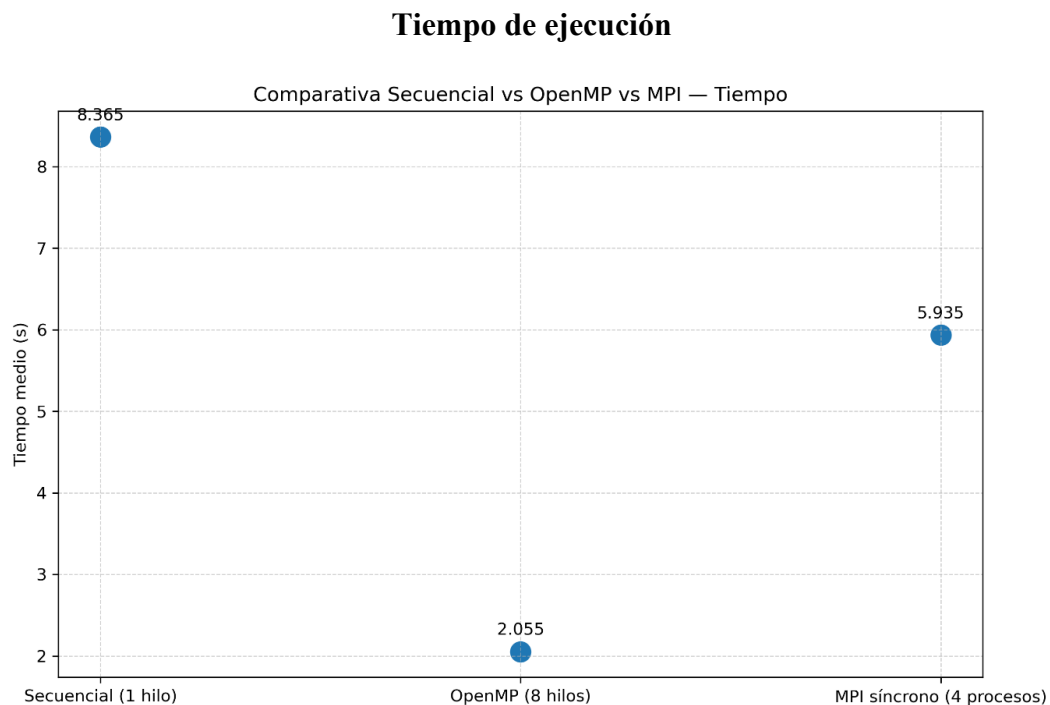
Le siguen de cerca las versiones colectiva y asíncrona, que logran altos valores de fitness a costa de un pequeño incremento en el tiempo de ejecución.

La versión con MPI_Pack/MPI_Unpack, en cambio, es la menos eficiente, al combinar un tiempo elevado con un fitness mediocre.

En definitiva, el **modelo síncrono con tipo derivado** se consolida como la opción más eficiente y rápida mientras que el colectivo ofrece la mejor calidad de solución. Esto demuestra que en ocasiones la sobreingeniería tiene más coste que beneficio y que la solución más compleja no tiene porqué ser la mejor. De hecho, la óptima es la configuración más simple.

Comparativa del algoritmo secuencial, paralelo con OpenMP y paralelo con MPI

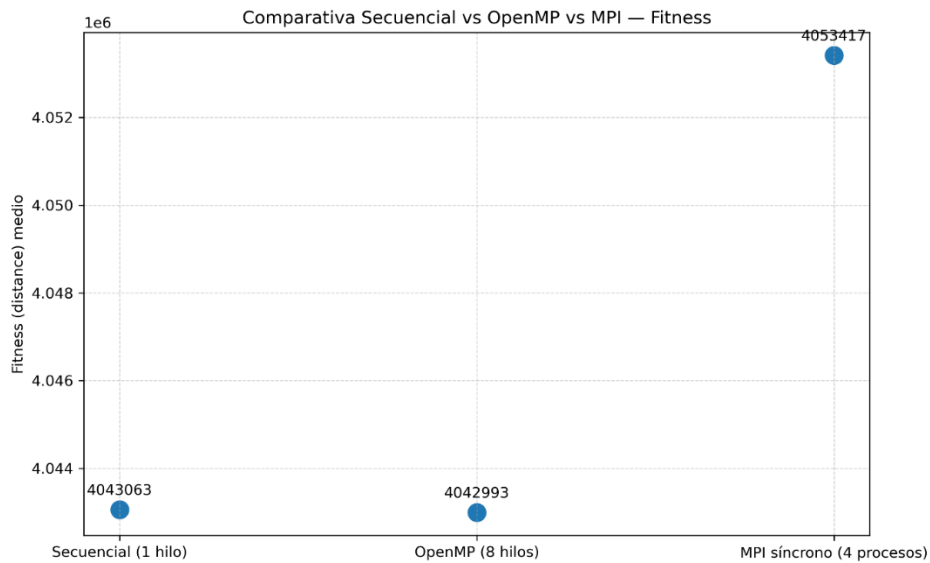
Una vez hemos obtenido la mejor configuración del algoritmo con MPI procedemos a compararlo con la versión en memoria compartida y secuencial.



Podemos observar como con OpenMP se ha obtenido el mejor tiempo de ejecución con bastante diferencia. Casi 4 segundos con respecto a la versión con MPI síncrono y más de 6 respecto al secuencial.

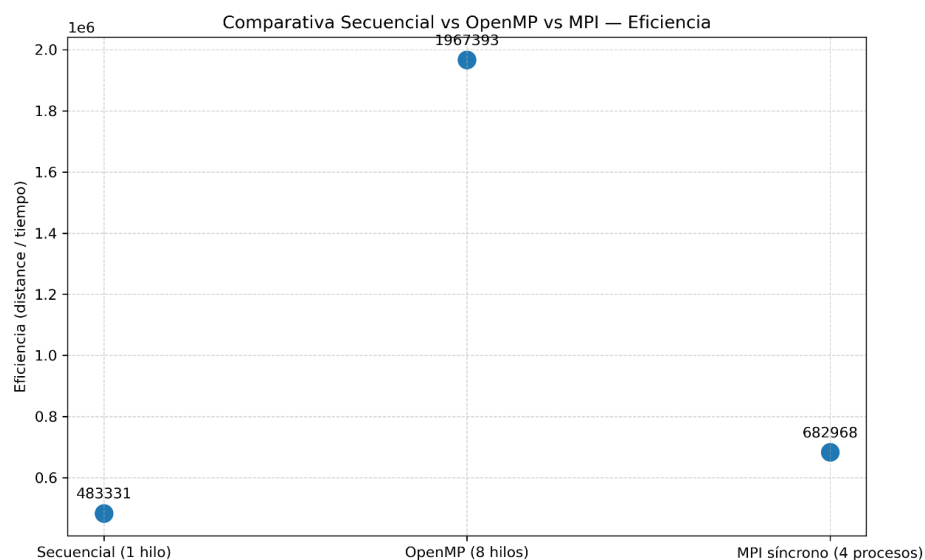
Está claro que el paradigma distribuido genera un coste adicional debido a las comunicaciones: envíos, recepciones y bloqueos. En OpenMP, al ser la memoria compartida, no existe este tipo de sobre coste. Además, la función fitness está paralelizada con reduction, lo que disminuye considerablemente el tiempo en computar la suma de las distancias de un individuo. Por si fuera poco, cruzar también está paralelizada. Estos dos factores son los que influyen con tanta notoriedad en la diferencia de tiempo entre OpenMP y MPI, por no hablar del coste adicional que suponen las migraciones o el mayor uso de memoria del algoritmo con MPI: reserva de búfferes de envío y recepción, etcétera.

Fitness



En cuanto al fitness, también se han obtenido los resultados esperados. OpenMP no sólo no mejora la calidad de la solución respecto al secuencial sino que la empeora ligeramente. En otras palabras, OpenMP es muy útil para obtener un tiempo de ejecución competente pero no tanto para el valor del fitness. Sin embargo, con MPI se obtiene la mejor solución con diferencia debido al reparto de la población entre los distintos procesos y su desarrollo paralelo evolutivo independiente junto a migraciones que permiten aumentar la diversidad genética convergiendo hacia una solución de mayor calidad.

Eficiencia



Finalmente, OpenMP es el que ofrece notablemente el balance óptimo entre el fitness y el tiempo de ejecución. Si bien con memoria compartida hemos obtenido el peor fitness, su tiempo de ejecución ha sido tan bajo que el índice de eficiencia es el mejor. Por tanto, **la versión paralela con OpenMP es la ganadora hasta ahora.**

Conclusión

En cuanto al reparto del trabajo se ha usado un documento compartido en Word para la redacción de esta memoria, así como la extensión LiveShare de Vscodé para trabajar simultáneamente en el desarrollo del código.

Ha sido interesante ver cómo en términos de eficiencia la versión síncrona con tipo de derivado ha sido la mejor respecto a la asíncrona y la colectiva, lo que pone de manifiesto que la solución más simple es a menudo la mejor.

Finalmente, nos ha sorprendido que OpenMP haya sido la mejor versión global hasta ahora. Pensábamos que, MPI, al mejorar bastante la calidad de la solución debido al planteamiento de las islas, obtendría un mejor índice de bondad pero no ha sido así. El tiempo de ejecución con OpenMP es demasiado competitivo como para que no compense la mejora del fitness de MPI.